

pyspark.sql module

Module Context

Important classes of Spark SQL and DataFrames:

- **pyspark.sql.SQLContext** Main entry point for **DataFrame** and SQL functionality.
- **pyspark.sql.DataFrame** A distributed collection of data grouped into named columns.
- **pyspark.sql.Column** A column expression in a **DataFrame**.
- **pyspark.sql.Row** A row of data in a **DataFrame**.
- **pyspark.sql.HiveContext** Main entry point for accessing data stored in Apache Hive.
- **pyspark.sql.GroupedData** Aggregation methods, returned by **DataFrame.groupBy()**.
- **pyspark.sql.DataFrameNaFunctions** Methods for handling missing data (null values).
- **pyspark.sql.DataFrameStatFunctions** Methods for statistics functionality.
- **pyspark.sql.functions** List of built-in functions available for **DataFrame**.
- **pyspark.sql.types** List of data types available.
- **pyspark.sql.Window** For working with window functions.

`class pyspark.sql.SQLContext(sparkContext, sqlContext=None)`

Main entry point for Spark SQL functionality.

A SQLContext can be used create **DataFrame**, register **DataFrame** as tables, execute SQL over tables, cache tables, and read parquet files.

Parameters:

- **sparkContext** – The **SparkContext** backing this SQLContext.
- **sqlContext** – An optional JVM Scala SQLContext. If set, we do not instantiate a new SQLContext in the JVM, instead we make all calls to this object.

`applySchema(rdd, schema)`

Note: Deprecated in 1.3, use `createDataFrame()` instead.

`cacheTable(tableName)`

Caches the specified table in-memory.

New in version 1.0.

`clearCache()`

Removes all cached tables from the in-memory cache.

New in version 1.3.

`createDataFrame(data, schema=None, samplingRatio=None)`

Creates a `DataFrame` from an RDD of `tuple/list`, `list` or `pandas.DataFrame`.

When `schema` is a list of column names, the type of each column will be inferred from `data`.

When `schema` is `None`, it will try to infer the schema (column names and types) from `data`, which should be an RDD of `Row`, or `namedtuple`, or `dict`.

If schema inference is needed, `samplingRatio` is used to determined the ratio of rows used for schema inference. The first row will be used if `samplingRatio` is `None`.

Parameters:

- **data** – an RDD of `Row/tuple/list/dict`, `list`, or `pandas.DataFrame`.
- **schema** – a `StructType` or list of column names. default `None`.
- **samplingRatio** – the sample ratio of rows used for inferring

Returns: `DataFrame`

```
>>> l = [('Alice', 1)]
>>> sqlContext.createDataFrame(l).collect()
[Row(_1=u'Alice', _2=1)]
>>> sqlContext.createDataFrame(l, ['name', 'age']).collect()
[Row(name=u'Alice', age=1)]
```

```
>>> d = [{'name': 'Alice', 'age': 1}]
>>> sqlContext.createDataFrame(d).collect()
[Row(age=1, name=u'Alice')]
```

```
>>> rdd = sc.parallelize(l)
>>> sqlContext.createDataFrame(rdd).collect()
[Row(_1=u'Alice', _2=1)]
>>> df = sqlContext.createDataFrame(rdd, ['name', 'age'])
>>> df.collect()
[Row(name=u'Alice', age=1)]
```

```
>>> from pyspark.sql import Row
>>> Person = Row('name', 'age')
>>> person = rdd.map(lambda r: Person(*r))
>>> df2 = sqlContext.createDataFrame(person)
>>> df2.collect()
[Row(name=u'Alice', age=1)]
```

```
>>> from pyspark.sql.types import *
>>> schema = StructType([
...     StructField("name", StringType(), True),
...     StructField("age", IntegerType(), True)])
>>> df3 = sqlContext.createDataFrame(rdd, schema)
>>> df3.collect()
[Row(name=u'Alice', age=1)]
```

```
>>> sqlContext.createDataFrame(df.toPandas()).collect()
[Row(name=u'Alice', age=1)]
>>> sqlContext.createDataFrame(pandas.DataFrame([[1, 2]])).collect()
[Row(0=1, 1=2)]
```

New in version 1.3.

createExternalTable(*tableName*, *path=None*, *source=None*, *schema=None*, ***options*)

Creates an external table based on the dataset in a data source.

It returns the DataFrame associated with the external table.

The data source is specified by the *source* and a set of options. If *source* is not specified, the default data source configured by `spark.sql.sources.default` will be used.

Optionally, a schema can be provided as the schema of the returned **DataFrame** and created external table.

Returns: **DataFrame**

New in version 1.3.

dropTempTable(*tableName*)

Remove the temp table from catalog.

```
>>> sqlContext.registerDataFrameAsTable(df, "table1")
>>> sqlContext.dropTempTable("table1")
```

New in version 1.6.

getConf(*key*, *defaultValue*)

Returns the value of Spark SQL configuration property for the given key.

If the key is not set, returns *defaultValue*.

New in version 1.3.

classmethod **getOrCreate**(*sc*)

Get the existing SQLContext or create a new one with given SparkContext.

Parameters: **sc** – SparkContext

New in version 1.6.

inferSchema(*rdd*, *samplingRatio=None*)

Note: Deprecated in 1.3, use **createDataFrame()** instead.

jsonFile(path, schema=None, samplingRatio=1.0)

Loads a text file storing one JSON object per line as a **DataFrame**.

Note: Deprecated in 1.4, use **DataFrameReader.json()** instead.

```
>>> sqlContext.jsonFile('python/test_support/sql/people.json').dtypes
[('age', 'bigint'), ('name', 'string')]
```

jsonRDD(rdd, schema=None, samplingRatio=1.0)

Loads an RDD storing one JSON object per string as a **DataFrame**.

If the schema is provided, applies the given schema to this JSON dataset. Otherwise, it samples the dataset with ratio `samplingRatio` to determine the schema.

```
>>> df1 = sqlContext.jsonRDD(json)
>>> df1.first()
Row(field1=1, field2=u'row1', field3=Row(field4=11, field5=None), field6=None)
```

```
>>> df2 = sqlContext.jsonRDD(json, df1.schema)
>>> df2.first()
Row(field1=1, field2=u'row1', field3=Row(field4=11, field5=None), field6=None)
```

```
>>> from pyspark.sql.types import *
>>> schema = StructType([
...     StructField("field2", StringType()),
...     StructField("field3",
...                 StructType([StructField("field5", ArrayType(IntegerType()))
... ])]
>>> df3 = sqlContext.jsonRDD(json, schema)
>>> df3.first()
Row(field2=u'row1', field3=Row(field5=None))
```

New in version 1.0.

load(path=None, source=None, schema=None, **options)

Returns the dataset in a data source as a **DataFrame**.

Note: Deprecated in 1.4, use **DataFrameReader.load()** instead.

newSession()

Returns a new **SQLContext** as new session, that has separate **SQLConf**, registered temporary tables and UDFs, but shared **SparkContext** and table cache.

New in version 1.6.

parquetFile(*paths)

Loads a Parquet file, returning the result as a **DataFrame**.

Note: Deprecated in 1.4, use **DataFrameReader.parquet()** instead.

```
>>> sqlContext.parquetFile('python/test_support/sql/parquet_partitioned').dtypes
[('name', 'string'), ('year', 'int'), ('month', 'int'), ('day', 'int')]
```

range(start, end=None, step=1, numPartitions=None)

Create a **DataFrame** with single LongType column named *id*, containing elements in a range from *start* to *end* (exclusive) with step value *step*.

Parameters:

- **start** – the start value
- **end** – the end value (exclusive)
- **step** – the incremental step (default: 1)
- **numPartitions** – the number of partitions of the DataFrame

Returns: **DataFrame**

```
>>> sqlContext.range(1, 7, 2).collect()
[Row(id=1), Row(id=3), Row(id=5)]
```

If only one argument is specified, it will be used as the end value.

```
>>> sqlContext.range(3).collect()
[Row(id=0), Row(id=1), Row(id=2)]
```

New in version 1.4.

read

Returns a **DataFrameReader** that can be used to read data in as a **DataFrame**.

Returns: **DataFrameReader**

New in version 1.4.

registerDataFrameAsTable(df, tableName)

Registers the given **DataFrame** as a temporary table in the catalog.

Temporary tables exist only during the lifetime of this instance of **SQLContext**.

```
>>> sqlContext.registerDataFrameAsTable(df, "table1")
```

New in version 1.3.

registerFunction(*name*, *f*, *returnType=StringType*)

Registers a python function (including lambda function) as a UDF so it can be used in SQL statements.

In addition to a name and the function itself, the return type can be optionally specified. When the return type is not given it default to a string and conversion will automatically be done. For any other return type, the produced object must match the specified type.

Parameters:

- **name** – name of the UDF
- **f** – python function
- **returnType** – a `DataType` object

```
>>> sqlContext.registerFunction("stringLengthString", lambda x: len(x))
>>> sqlContext.sql("SELECT stringLengthString('test']").collect()
[Row(_c0=u'4')]
```

```
>>> from pyspark.sql.types import IntegerType
>>> sqlContext.registerFunction("stringLengthInt", lambda x: len(x), IntegerType())
>>> sqlContext.sql("SELECT stringLengthInt('test']").collect()
[Row(_c0=4)]
```

```
>>> from pyspark.sql.types import IntegerType
>>> sqlContext.udf.register("stringLengthInt", lambda x: len(x), IntegerType())
>>> sqlContext.sql("SELECT stringLengthInt('test']").collect()
[Row(_c0=4)]
```

New in version 1.2.

setConf(*key*, *value*)

Sets the given Spark SQL configuration property.

New in version 1.3.

sql(*sqlQuery*)

Returns a `DataFrame` representing the result of the given query.

Returns: `DataFrame`

```
>>> sqlContext.registerDataFrameAsTable(df, "table1")
>>> df2 = sqlContext.sql("SELECT field1 AS f1, field2 as f2 from table1")
>>> df2.collect()
[Row(f1=1, f2=u'row1'), Row(f1=2, f2=u'row2'), Row(f1=3, f2=u'row3')]
```

New in version 1.0.

table(*tableName*)

Returns the specified table as a **DataFrame**.

Returns: **DataFrame**

```
>>> sqlContext.registerDataFrameAsTable(df, "table1")
>>> df2 = sqlContext.table("table1")
>>> sorted(df.collect()) == sorted(df2.collect())
True
```

New in version 1.0.

tableNames(*dbName=None*)

Returns a list of names of tables in the database *dbName*.

Parameters: **dbName** – string, name of the database to use. Default to the current database.

Returns: list of table names, in string

```
>>> sqlContext.registerDataFrameAsTable(df, "table1")
>>> "table1" in sqlContext.tableNames()
True
>>> "table1" in sqlContext.tableNames("db")
True
```

New in version 1.3.

tables(*dbName=None*)

Returns a **DataFrame** containing names of tables in the given database.

If *dbName* is not specified, the current database will be used.

The returned **DataFrame** has two columns: *tableName* and *isTemporary* (a column with **BooleanType** indicating if a table is a temporary one or not).

Parameters: **dbName** – string, name of the database to use.

Returns: **DataFrame**

```
>>> sqlContext.registerDataFrameAsTable(df, "table1")
>>> df2 = sqlContext.tables()
>>> df2.filter("tableName = 'table1']").first()
Row(tableName=u'table1', isTemporary=True)
```

New in version 1.3.

udf

Returns a **UDFRegistration** for UDF registration.

Returns: **UDFRegistration**

New in version 1.3.1.

uncacheTable(*tableName*)

Removes the specified table from the in-memory cache.

New in version 1.0.

class pyspark.sql.**HiveContext**(*sparkContext*, *hiveContext=None*)

A variant of Spark SQL that integrates with data stored in Hive.

Configuration for Hive is read from `hive-site.xml` on the classpath. It supports running both SQL and HiveQL commands.

Parameters:

- **sparkContext** – The SparkContext to wrap.
- **hiveContext** – An optional JVM Scala HiveContext. If set, we do not instantiate a new **HiveContext** in the JVM, instead we make all calls to this object.

refreshTable(*tableName*)

Invalidate and refresh all the cached the metadata of the given table. For performance reasons, Spark SQL or the external data source library it uses might cache certain metadata about a table, such as the location of blocks. When those change outside of Spark SQL, users should call this function to invalidate the cache.

class pyspark.sql.**DataFrame**(*jdf*, *sql_ctx*)

A distributed collection of data grouped into named columns.

A **DataFrame** is equivalent to a relational table in Spark SQL, and can be created using various functions in **SQLContext**:

```
people = sqlContext.read.parquet("...")
```

Once created, it can be manipulated using the various domain-specific-language (DSL) functions defined in: **DataFrame**, **Column**.

To select a column from the data frame, use the apply method:

```
ageCol = people.age
```

A more concrete example:

```
# To create DataFrame using SQLContext
people = sqlContext.read.parquet("...")
department = sqlContext.read.parquet("...")

people.filter(people.age > 30).join(department, people.deptId == department.id))
```


Note: Experimental

New in version 1.3.

agg(*exprs)

Aggregate on the entire **DataFrame** without groups (shorthand for `df.groupBy.agg()`).

```
>>> df.agg({"age": "max"}).collect()
[Row(max(age)=5)]
>>> from pyspark.sql import functions as F
>>> df.agg(F.min(df.age)).collect()
[Row(min(age)=2)]
```

New in version 1.3.

alias(alias)

Returns a new **DataFrame** with an alias set.

```
>>> from pyspark.sql.functions import *
>>> df_as1 = df.alias("df_as1")
>>> df_as2 = df.alias("df_as2")
>>> joined_df = df_as1.join(df_as2, col("df_as1.name") == col("df_as2.name"),
>>> joined_df.select(col("df_as1.name"), col("df_as2.name"), col("df_as2.age")
[Row(name=u'Alice', name=u'Alice', age=2), Row(name=u'Bob', name=u'Bob', age=
```

New in version 1.3.

cache()

Persists with the default storage level (**MEMORY_ONLY_SER**).

New in version 1.3.

coalesce(numPartitions)

Returns a new **DataFrame** that has exactly *numPartitions* partitions.

Similar to coalesce defined on an **RDD**, this operation results in a narrow dependency, e.g. if you go from 1000 partitions to 100 partitions, there will not be a shuffle, instead each of the 100 new partitions will claim 10 of the current partitions.

```
>>> df.coalesce(1).rdd.getNumPartitions()
1
```

New in version 1.4.

collect()

Returns all the records as a list of **Row**.

```
>>> df.collect()
[Row(age=2, name=u'Alice'), Row(age=5, name=u'Bob')]
```

New in version 1.3.

columns

Returns all column names as a list.

```
>>> df.columns
['age', 'name']
```

New in version 1.3.

corr(col1, col2, method=None)

Calculates the correlation of two columns of a `DataFrame` as a double value. Currently only supports the Pearson Correlation Coefficient. `DataFrame.corr()` and `DataFrameStatFunctions.corr()` are aliases of each other.

Parameters:

- **col1** – The name of the first column
- **col2** – The name of the second column
- **method** – The correlation method. Currently only supports “pearson”

New in version 1.4.

count()

Returns the number of rows in this `DataFrame`.

```
>>> df.count()
2
```

New in version 1.3.

cov(col1, col2)

Calculate the sample covariance for the given columns, specified by their names, as a double value. `DataFrame.cov()` and `DataFrameStatFunctions.cov()` are aliases.

Parameters:

- **col1** – The name of the first column
- **col2** – The name of the second column

New in version 1.4.

crosstab(col1, col2)

Computes a pair-wise frequency table of the given columns. Also known as a contingency table. The number of distinct values for each column should be less than 1e4. At most 1e6 non-zero pair frequencies will be returned. The first column of each row will be the distinct values of *col1* and the column names will be the distinct values


```
+-----+-----+-----+
| count|                2|      2|
| mean|                3.5|   null|
| stddev|2.1213203435596424| null|
| min|                2| Alice|
| max|                5|   Bob|
+-----+-----+-----+
```

New in version 1.3.1.

distinct()

Returns a new **DataFrame** containing the distinct rows in this **DataFrame**.

```
>>> df.distinct().count()
2
```

New in version 1.3.

drop(col)

Returns a new **DataFrame** that drops the specified column.

Parameters: **col** – a string name of the column to drop, or a **Column** to drop.

```
>>> df.drop('age').collect()
[Row(name=u'Alice'), Row(name=u'Bob')]
```

```
>>> df.drop(df.age).collect()
[Row(name=u'Alice'), Row(name=u'Bob')]
```

```
>>> df.join(df2, df.name == df2.name, 'inner').drop(df.name).collect()
[Row(age=5, height=85, name=u'Bob')]
```

```
>>> df.join(df2, df.name == df2.name, 'inner').drop(df2.name).collect()
[Row(age=5, name=u'Bob', height=85)]
```

New in version 1.4.

dropDuplicates(subset=None)

Return a new **DataFrame** with duplicate rows removed, optionally only considering certain columns.

drop_duplicates() is an alias for **dropDuplicates()**.

```
>>> from pyspark.sql import Row
>>> df = sc.parallelize([Row(name='Alice', age=5, height=80),
>>> df.dropDuplicates().show()
+---+-----+-----+
|age|height| name|
```

```
+---+-----+-----+
|  5|      80|Alice|
| 10|      80|Alice|
+---+-----+-----+
```

```
>>> df.dropDuplicates(['name', 'height']).show()
+---+-----+-----+
|age|height| name|
+---+-----+-----+
|  5|      80|Alice|
+---+-----+-----+
```

New in version 1.4.

`drop_duplicates(subset=None)`

Return a new **DataFrame** with duplicate rows removed, optionally only considering certain columns.

`drop_duplicates()` is an alias for `dropDuplicates()`.

```
>>> from pyspark.sql import Row
>>> df = sc.parallelize([Row(name='Alice', age=5, height=80),
>>> df.dropDuplicates().show()
+---+-----+-----+
|age|height| name|
+---+-----+-----+
|  5|      80|Alice|
| 10|      80|Alice|
+---+-----+-----+
```

```
>>> df.dropDuplicates(['name', 'height']).show()
+---+-----+-----+
|age|height| name|
+---+-----+-----+
|  5|      80|Alice|
+---+-----+-----+
```

New in version 1.4.

`dropna(how='any', thresh=None, subset=None)`

Returns a new **DataFrame** omitting rows with null values. `DataFrame.dropna()` and `DataFrameNaFunctions.drop()` are aliases of each other.

- Parameters:**
- **how** – ‘any’ or ‘all’. If ‘any’, drop a row if it contains any nulls. If ‘all’, drop a row only if all its values are null.
 - **thresh** – int, default None. If specified, drop rows that have less than *thresh* non-null values. This overwrites the *how* parameter.
 - **subset** – optional list of column names to consider.

```
>>> df4.na.drop().show()
+---+-----+-----+
|age|height| name|
+---+-----+-----+
| 10|    80|Alice|
+---+-----+-----+
```

New in version 1.3.1.

dtypes

Returns all column names and their data types as a list.

```
>>> df.dtypes
[('age', 'int'), ('name', 'string')]
```

New in version 1.3.

explain(extended=False)

Prints the (logical and physical) plans to the console for debugging purpose.

Parameters: **extended** – boolean, default False. If False, prints only the physical plan.

```
>>> df.explain()
== Physical Plan ==
Scan ExistingRDD[age#0,name#1]
```

```
>>> df.explain(True)
== Parsed Logical Plan ==
...
== Analyzed Logical Plan ==
...
== Optimized Logical Plan ==
...
== Physical Plan ==
...
```

New in version 1.3.

fillna(value, subset=None)

Replace null values, alias for `na.fill()`. `DataFrame.fillna()` and `DataFrameNaFunctions.fill()` are aliases of each other.

Parameters:

- **value** – int, long, float, string, or dict. Value to replace null values with. If the value is a dict, then *subset* is ignored and *value* must be a mapping from column name (string) to replacement value. The replacement value must be an int, long, float, or string.
- **subset** – optional list of column names to consider. Columns specified in subset that do not have matching data type are

ignored. For example, if *value* is a string, and subset contains a non-string column, then the non-string column is simply ignored.

```
>>> df4.na.fill(50).show()
+---+-----+-----+
|age|height| name|
+---+-----+-----+
| 10|    80|Alice|
|  5|    50|  Bob|
| 50|    50|  Tom|
| 50|    50| null|
+---+-----+-----+
```

```
>>> df4.na.fill({'age': 50, 'name': 'unknown'}).show()
+---+-----+-----+
|age|height|  name|
+---+-----+-----+
| 10|    80| Alice|
|  5|   null|  Bob|
| 50|   null|  Tom|
| 50|   null|unknown|
+---+-----+-----+
```

New in version 1.3.1.

filter(condition)

Filters rows using the given condition.

where() is an alias for **filter()**.

Parameters: **condition** – a Column of types `BooleanType` or a string of SQL expression.

```
>>> df.filter(df.age > 3).collect()
[Row(age=5, name=u'Bob')]
>>> df.where(df.age == 2).collect()
[Row(age=2, name=u'Alice')]
```

```
>>> df.filter("age > 3").collect()
[Row(age=5, name=u'Bob')]
>>> df.where("age = 2").collect()
[Row(age=2, name=u'Alice')]
```

New in version 1.3.

first()

Returns the first row as a **Row**.

```
>>> df.first()
Row(age=2, name=u'Alice')
```

New in version 1.3.

flatMap(*f*)

Returns a new **RDD** by first applying the *f* function to each **Row**, and then flattening the results.

This is a shorthand for `df.rdd.flatMap()`.

```
>>> df.flatMap(lambda p: p.name).collect()
[u'A', u'l', u'i', u'c', u'e', u'B', u'o', u'b']
```

New in version 1.3.

foreach(*f*)

Applies the *f* function to all **Row** of this **DataFrame**.

This is a shorthand for `df.rdd.foreach()`.

```
>>> def f(person):
...     print(person.name)
>>> df.foreach(f)
```

New in version 1.3.

foreachPartition(*f*)

Applies the *f* function to each partition of this **DataFrame**.

This a shorthand for `df.rdd.foreachPartition()`.

```
>>> def f(people):
...     for person in people:
...         print(person.name)
>>> df.foreachPartition(f)
```

New in version 1.3.

freqItems(*cols*, *support=None*)

Finding frequent items for columns, possibly with false positives. Using the frequent element count algorithm described in “<http://dx.doi.org/10.1145/762471.762473>, proposed by Karp, Schenker, and Papadimitriou”. **DataFrame.freqItems()** and **DataFrameStatFunctions.freqItems()** are aliases.

Note: This function is meant for exploratory data analysis, as we make no guarantee about the backward compatibility of the schema of the resulting **DataFrame**.

Parameters: • **cols** – Names of the columns to calculate frequent items for as a

list or tuple of strings.

- **support** – The frequency with which to consider an item ‘frequent’. Default is 1%. The support must be greater than 1e-4.

New in version 1.4.

groupBy(*cols)

Groups the **DataFrame** using the specified columns, so we can run aggregation on them. See **GroupedData** for all the available aggregate functions.

groupby() is an alias for **groupBy()**.

Parameters: **cols** – list of columns to group by. Each element should be a column name (string) or an expression (**Column**).

```
>>> df.groupBy().avg().collect()
[Row(avg(age)=3.5)]
>>> df.groupBy('name').agg({'age': 'mean'}).collect()
[Row(name=u'Alice', avg(age)=2.0), Row(name=u'Bob', avg(age)=5.0)]
>>> df.groupBy(df.name).avg().collect()
[Row(name=u'Alice', avg(age)=2.0), Row(name=u'Bob', avg(age)=5.0)]
>>> df.groupBy(['name', df.age]).count().collect()
[Row(name=u'Bob', age=5, count=1), Row(name=u'Alice', age=2, count=1)]
```

New in version 1.3.

groupby(*cols)

Groups the **DataFrame** using the specified columns, so we can run aggregation on them. See **GroupedData** for all the available aggregate functions.

groupby() is an alias for **groupBy()**.

Parameters: **cols** – list of columns to group by. Each element should be a column name (string) or an expression (**Column**).

```
>>> df.groupBy().avg().collect()
[Row(avg(age)=3.5)]
>>> df.groupBy('name').agg({'age': 'mean'}).collect()
[Row(name=u'Alice', avg(age)=2.0), Row(name=u'Bob', avg(age)=5.0)]
>>> df.groupBy(df.name).avg().collect()
[Row(name=u'Alice', avg(age)=2.0), Row(name=u'Bob', avg(age)=5.0)]
>>> df.groupBy(['name', df.age]).count().collect()
[Row(name=u'Bob', age=5, count=1), Row(name=u'Alice', age=2, count=1)]
```

New in version 1.3.

head(n=None)

Returns the first n rows.

Parameters: **n** – int, default 1. Number of rows to return.

Returns: If n is greater than 1, return a list of **Row**. If n is 1, return a single **Row**.

```
>>> df.head()
Row(age=2, name=u'Alice')
>>> df.head(1)
[Row(age=2, name=u'Alice')]
```

New in version 1.3.

insertInto(*tableName*, *overwrite=False*)

Inserts the contents of this **DataFrame** into the specified table.

Note: Deprecated in 1.4, use **DataFrameWriter.insertInto()** instead.

intersect(*other*)

Return a new **DataFrame** containing rows only in both this frame and another frame.

This is equivalent to *INTERSECT* in SQL.

New in version 1.3.

isLocal()

Returns True if the **collect()** and **take()** methods can be run locally (without any Spark executors).

New in version 1.3.

join(*other*, *on=None*, *how=None*)

Joins with another **DataFrame**, using the given join expression.

The following performs a full outer join between df1 and df2.

Parameters:

- **other** – Right side of the join
- **on** – a string for join column name, a list of column names, , a join expression (Column) or a list of Columns. If *on* is a string or a list of string indicating the name of the join column(s), the column(s) must exist on both sides, and this performs an inner equi-join.
- **how** – str, default 'inner'. One of *inner*, *outer*, *left_outer*, *right_outer*, *leftsemi*.

```
>>> df.join(df2, df.name == df2.name, 'outer').select(df.name, df2.height).collect()
[Row(name=None, height=80), Row(name=u'Alice', height=None), Row(name=u'Bob',
```

```
>>> cond = [df.name == df3.name, df.age == df3.age]
>>> df.join(df3, cond, 'outer').select(df.name, df3.age).collect()
[Row(name=u'Bob', age=5), Row(name=u'Alice', age=2)]
```

```
>>> df.join(df2, 'name').select(df.name, df2.height).collect()
```

```
[Row(name=u'Bob', height=85)]
```

```
>>> df.join(df4, ['name', 'age']).select(df.name, df.age).collect()
[Row(name=u'Bob', age=5)]
```

New in version 1.3.

limit(num)

Limits the result count to the number specified.

```
>>> df.limit(1).collect()
[Row(age=2, name=u'Alice')]
>>> df.limit(0).collect()
[]
```

New in version 1.3.

map(f)

Returns a new **RDD** by applying a the *f* function to each **Row**.

This is a shorthand for `df.rdd.map()`.

```
>>> df.map(lambda p: p.name).collect()
[u'Alice', u'Bob']
```

New in version 1.3.

mapPartitions(f, preservesPartitioning=False)

Returns a new **RDD** by applying the *f* function to each partition.

This is a shorthand for `df.rdd.mapPartitions()`.

```
>>> rdd = sc.parallelize([1, 2, 3, 4], 4)
>>> def f(iterator): yield 1
>>> rdd.mapPartitions(f).sum()
4
```

New in version 1.3.

na

Returns a **DataFrameNaFunctions** for handling missing values.

New in version 1.3.1.

orderBy(*cols, **kwargs)

Returns a new **DataFrame** sorted by the specified column(s).

Parameters:

- **cols** – list of **column** or column names to sort by.
- **ascending** – boolean or list of boolean (default True). Sort ascending vs. descending. Specify list for multiple sort orders. If a list is specified, length of the list must equal length of the *cols*.

```
>>> df.sort(df.age.desc()).collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
>>> df.sort("age", ascending=False).collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
>>> df.orderBy(df.age.desc()).collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
>>> from pyspark.sql.functions import *
>>> df.sort(asc("age")).collect()
[Row(age=2, name=u'Alice'), Row(age=5, name=u'Bob')]
>>> df.orderBy(desc("age"), "name").collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
>>> df.orderBy(["age", "name"], ascending=[0, 1]).collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
```

New in version 1.3.

persist(storageLevel=StorageLevel(False, True, False, False, 1))

Sets the storage level to persist its values across operations after the first time it is computed. This can only be used to assign a new storage level if the RDD does not have a storage level set yet. If no storage level is specified defaults to (MEMORY_ONLY_SER).

New in version 1.3.

printSchema()

Prints out the schema in the tree format.

```
>>> df.printSchema()
root
|-- age: integer (nullable = true)
|-- name: string (nullable = true)
```

New in version 1.3.

randomSplit(weights, seed=None)

Randomly splits this **DataFrame** with the provided weights.

Parameters:

- **weights** – list of doubles as weights with which to split the DataFrame. Weights will be normalized if they don't sum up to 1.0.
- **seed** – The seed for sampling.

```
>>> splits = df4.randomSplit([1.0, 2.0], 24)
>>> splits[0].count()
1
```

```
>>> splits[1].count()
3
```

New in version 1.4.

rdd

Returns the content as an `pyspark.RDD` of `Row`.

New in version 1.3.

registerAsTable(name)

Note: Deprecated in 1.4, use `registerTempTable()` instead.

registerTempTable(name)

Registers this RDD as a temporary table using the given name.

The lifetime of this temporary table is tied to the `SQLContext` that was used to create this `DataFrame`.

```
>>> df.registerTempTable("people")
>>> df2 = sqlContext.sql("select * from people")
>>> sorted(df.collect()) == sorted(df2.collect())
True
```

New in version 1.3.

repartition(numPartitions, *cols)

Returns a new `DataFrame` partitioned by the given partitioning expressions. The resulting `DataFrame` is hash partitioned.

`numPartitions` can be an int to specify the target number of partitions or a `Column`. If it is a `Column`, it will be used as the first partitioning column. If not specified, the default number of partitions is used.

Changed in version 1.6: Added optional arguments to specify the partitioning columns. Also made `numPartitions` optional if partitioning columns are specified.

```
>>> df.repartition(10).rdd.getNumPartitions()
10
>>> data = df.unionAll(df).repartition("age")
>>> data.show()
+---+-----+
|age| name|
+---+-----+
|  2|Alice|
|  2|Alice|
|  5|  Bob|
|  5|  Bob|
+---+-----+
>>> data = data.repartition(7, "age")
```

```

>>> data.show()
+---+-----+
|age| name|
+---+-----+
|  5|  Bob|
|  5|  Bob|
|  2|Alice|
|  2|Alice|
+---+-----+
>>> data.rdd.getNumPartitions()
7
>>> data = data.repartition("name", "age")
>>> data.show()
+---+-----+
|age| name|
+---+-----+
|  5|  Bob|
|  5|  Bob|
|  2|Alice|
|  2|Alice|
+---+-----+

```

New in version 1.3.

`replace(to_replace, value, subset=None)`

Returns a new `DataFrame` replacing a value with another value. `DataFrame.replace()` and `DataFrameNaFunctions.replace()` are aliases of each other.

- Parameters:**
- **to_replace** – int, long, float, string, or list. Value to be replaced. If the value is a dict, then *value* is ignored and *to_replace* must be a mapping from column name (string) to replacement value. The value to be replaced must be an int, long, float, or string.
 - **value** – int, long, float, string, or list. Value to use to replace holes. The replacement value must be an int, long, float, or string. If *value* is a list or tuple, *value* should be of the same length with *to_replace*.
 - **subset** – optional list of column names to consider. Columns specified in subset that do not have matching data type are ignored. For example, if *value* is a string, and subset contains a non-string column, then the non-string column is simply ignored.

```

>>> df4.na.replace(10, 20).show()
+---+-----+-----+
| age|height| name|
+---+-----+-----+
| 20|    80|Alice|
|  5|   null|  Bob|
| null|  null|  Tom|
| null|  null| null|
+---+-----+-----+

```

```

>>> df4.na.replace(['Alice', 'Bob'], ['A', 'B'], 'name').show()
+---+-----+-----+
| age|height|name|
+---+-----+-----+

```

```
+---+---+---+
| 10|  80|  A|
|  5| null|  B|
| null| null| Tom|
| null| null| null|
+---+---+---+
```

New in version 1.4.

rollup(*cols)

Create a multi-dimensional rollup for the current **DataFrame** using the specified columns, so we can run aggregation on them.

```
>>> df.rollup('name', df.age).count().show()
+---+---+---+
| name| age|count|
+---+---+---+
| Alice| null|    1|
|  Bob|   5|    1|
|  Bob| null|    1|
| null| null|    2|
| Alice|  2|    1|
+---+---+---+
```

New in version 1.4.

sample(withReplacement, fraction, seed=None)

Returns a sampled subset of this **DataFrame**.

```
>>> df.sample(False, 0.5, 42).count()
2
```

New in version 1.3.

sampleBy(col, fractions, seed=None)

Returns a stratified sample without replacement based on the fraction given on each stratum.

- Parameters:**
- **col** – column that defines strata
 - **fractions** – sampling fraction for each stratum. If a stratum is not specified, we treat its fraction as zero.
 - **seed** – random seed

Returns: a new **DataFrame** that represents the stratified sample

```
>>> from pyspark.sql.functions import col
>>> dataset = sqlContext.range(0, 100).select((col("id") % 3).alias("key"))
>>> sampled = dataset.sampleBy("key", fractions={0: 0.1, 1: 0.2}, seed=0)
>>> sampled.groupBy("key").count().orderBy("key").show()
+---+---+
|key|count|
+---+---+
```

```
| 0 | 5 |
| 1 | 9 |
+---+---+
```

New in version 1.5.

save(*path=None, source=None, mode='error', **options*)

Saves the contents of the **DataFrame** to a data source.

Note: Deprecated in 1.4, use **DataFrameWriter.save()** instead.

New in version 1.3.

saveAsParquetFile(*path*)

Saves the contents as a Parquet file, preserving the schema.

Note: Deprecated in 1.4, use **DataFrameWriter.parquet()** instead.

saveAsTable(*tableName, source=None, mode='error', **options*)

Saves the contents of this **DataFrame** to a data source as a table.

Note: Deprecated in 1.4, use **DataFrameWriter.saveAsTable()** instead.

schema

Returns the schema of this **DataFrame** as a **types.StructType**.

```
>>> df.schema
StructType(List(StructField(age,IntegerType,true),StructField(name,StringType
◀ ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━▶ ▶
```

New in version 1.3.

select(**cols*)

Projects a set of expressions and returns a new **DataFrame**.

Parameters: **cols** – list of column names (string) or expressions (**Column**). If one of the column names is '*', that column is expanded to include all columns in the current **DataFrame**.

```
>>> df.select('*').collect()
[Row(age=2, name=u'Alice'), Row(age=5, name=u'Bob')]
>>> df.select('name', 'age').collect()
[Row(name=u'Alice', age=2), Row(name=u'Bob', age=5)]
>>> df.select(df.name, (df.age + 10).alias('age')).collect()
[Row(name=u'Alice', age=12), Row(name=u'Bob', age=15)]
```

New in version 1.3.

selectExpr(*expr)

Projects a set of SQL expressions and returns a new **DataFrame**.

This is a variant of **select()** that accepts SQL expressions.

```
>>> df.selectExpr("age * 2", "abs(age)").collect()
[Row((age * 2)=4, abs(age)=2), Row((age * 2)=10, abs(age)=5)]
```

New in version 1.3.

show(n=20, truncate=True)

Prints the first n rows to the console.

Parameters:

- **n** – Number of rows to show.
- **truncate** – Whether truncate long strings and align cells right.

```
>>> df
DataFrame[age: int, name: string]
>>> df.show()
+---+-----+
|age| name|
+---+-----+
|  2|Alice|
|  5|  Bob|
+---+-----+
```

New in version 1.3.

sort(*cols, **kwargs)

Returns a new **DataFrame** sorted by the specified column(s).

Parameters:

- **cols** – list of **Column** or column names to sort by.
- **ascending** – boolean or list of boolean (default True). Sort ascending vs. descending. Specify list for multiple sort orders. If a list is specified, length of the list must equal length of the *cols*.

```
>>> df.sort(df.age.desc()).collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
>>> df.sort("age", ascending=False).collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
>>> df.orderBy(df.age.desc()).collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
>>> from pyspark.sql.functions import *
>>> df.sort(asc("age")).collect()
[Row(age=2, name=u'Alice'), Row(age=5, name=u'Bob')]
>>> df.orderBy(desc("age"), "name").collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
>>> df.orderBy(["age", "name"], ascending=[0, 1]).collect()
[Row(age=5, name=u'Bob'), Row(age=2, name=u'Alice')]
```

New in version 1.3.

sortWithinPartitions(*cols, **kwargs)

Returns a new **DataFrame** with each partition sorted by the specified column(s).

- Parameters:**
- **cols** – list of **Column** or column names to sort by.
 - **ascending** – boolean or list of boolean (default True). Sort ascending vs. descending. Specify list for multiple sort orders. If a list is specified, length of the list must equal length of the *cols*.

```
>>> df.sortWithinPartitions("age", ascending=False).show()
+---+-----+
|age| name|
+---+-----+
|  2|Alice|
|  5|  Bob|
+---+-----+
```

New in version 1.6.

stat

Returns a **DataFrameStatFunctions** for statistic functions.

New in version 1.4.

subtract(other)

Return a new **DataFrame** containing rows in this frame but not in another frame.

This is equivalent to *EXCEPT* in SQL.

New in version 1.3.

take(num)

Returns the first num rows as a **list** of **Row**.

```
>>> df.take(2)
[Row(age=2, name=u'Alice'), Row(age=5, name=u'Bob')]
```

New in version 1.3.

toDF(*cols)

Returns a new class:*DataFrame* that with new specified column names

- Parameters:** **cols** – list of new column names (string)

```
>>> df.toDF('f1', 'f2').collect()
[Row(f1=2, f2=u'Alice'), Row(f1=5, f2=u'Bob')]
```

toJSON(use_unicode=True)

Converts a **DataFrame** into a **RDD** of string.

Each row is turned into a JSON document as one element in the returned RDD.

```
>>> df.toJSON().first()
u'{"age":2,"name":"Alice"}
```

New in version 1.3.

toPandas()

Returns the contents of this **DataFrame** as Pandas `pandas.DataFrame`.

This is only available if Pandas is installed and available.

```
>>> df.toPandas()
   age  name
0    2  Alice
1    5   Bob
```

New in version 1.3.

unionAll(*other*)

Return a new **DataFrame** containing union of rows in this frame and another frame.

This is equivalent to *UNION ALL* in SQL.

New in version 1.3.

unpersist(*blocking=True*)

Marks the **DataFrame** as non-persistent, and remove all blocks for it from memory and disk.

New in version 1.3.

where(*condition*)

Filters rows using the given condition.

where() is an alias for **filter()**.

Parameters: **condition** – a **Column** of types **BooleanType** or a string of SQL expression.

```
>>> df.filter(df.age > 3).collect()
[Row(age=5, name=u'Bob')]
>>> df.where(df.age == 2).collect()
[Row(age=2, name=u'Alice')]
```

```
>>> df.filter("age > 3").collect()
[Row(age=5, name=u'Bob')]
>>> df.where("age = 2").collect()
[Row(age=2, name=u'Alice')]
```

New in version 1.3.

withColumn(colName, col)

Returns a new **DataFrame** by adding a column or replacing the existing column that has the same name.

Parameters:

- **colName** – string, name of the new column.
- **col** – a **Column** expression for the new column.

```
>>> df.withColumn('age2', df.age + 2).collect()
[Row(age=2, name=u'Alice', age2=4), Row(age=5, name=u'Bob', age2=7)]
```

New in version 1.3.

withColumnRenamed(existing, new)

Returns a new **DataFrame** by renaming an existing column.

Parameters:

- **existing** – string, name of the existing column to rename.
- **col** – string, new name of the column.

```
>>> df.withColumnRenamed('age', 'age2').collect()
[Row(age2=2, name=u'Alice'), Row(age2=5, name=u'Bob')]
```

New in version 1.3.

write

Interface for saving the content of the **DataFrame** out into external storage.

Returns: **DataFrameWriter**

New in version 1.4.

class pyspark.sql.GrouppedData(jdf, sql_ctx)

A set of methods for aggregations on a **DataFrame**, created by **DataFrame.groupBy()**.

Note: Experimental

New in version 1.3.

agg(*exprs)

Compute aggregates and returns the result as a **DataFrame**.

The available aggregate functions are *avg*, *max*, *min*, *sum*, *count*.

If *exprs* is a single **dict** mapping from string to string, then the key is the column to perform aggregation on, and the value is the aggregate function.

Alternatively, `exprs` can also be a list of aggregate `Column` expressions.

Parameters: `exprs` – a dict mapping from column name (string) to aggregate functions (string), or a list of `Column`.

```
>>> gdf = df.groupBy(df.name)
>>> gdf.agg({"*": "count"}).collect()
[Row(name=u'Alice', count(1)=1), Row(name=u'Bob', count(1)=1)]
```

```
>>> from pyspark.sql import functions as F
>>> gdf.agg(F.min(df.age)).collect()
[Row(name=u'Alice', min(age)=2), Row(name=u'Bob', min(age)=5)]
```

New in version 1.3.

`avg(*args)`

Computes average values for each numeric columns for each group.

`mean()` is an alias for `avg()`.

Parameters: `cols` – list of column names (string). Non-numeric columns are ignored.

```
>>> df.groupBy().avg('age').collect()
[Row(avg(age)=3.5)]
>>> df3.groupBy().avg('age', 'height').collect()
[Row(avg(age)=3.5, avg(height)=82.5)]
```

New in version 1.3.

`count()`

Counts the number of records for each group.

```
>>> df.groupBy(df.age).count().collect()
[Row(age=2, count=1), Row(age=5, count=1)]
```

New in version 1.3.

`max(*args)`

Computes the max value for each numeric columns for each group.

```
>>> df.groupBy().max('age').collect()
[Row(max(age)=5)]
>>> df3.groupBy().max('age', 'height').collect()
[Row(max(age)=5, max(height)=85)]
```

New in version 1.3.

mean(*args)

Computes average values for each numeric columns for each group.

mean() is an alias for **avg()**.

Parameters: **cols** – list of column names (string). Non-numeric columns are ignored.

```
>>> df.groupBy().mean('age').collect()
[Row(avg(age)=3.5)]
>>> df3.groupBy().mean('age', 'height').collect()
[Row(avg(age)=3.5, avg(height)=82.5)]
```

New in version 1.3.

min(*args)

Computes the min value for each numeric column for each group.

Parameters: **cols** – list of column names (string). Non-numeric columns are ignored.

```
>>> df.groupBy().min('age').collect()
[Row(min(age)=2)]
>>> df3.groupBy().min('age', 'height').collect()
[Row(min(age)=2, min(height)=80)]
```

New in version 1.3.

pivot(pivot_col, values=None)

Pivots a column of the current [[DataFrame]] and perform the specified aggregation. There are two versions of pivot function: one that requires the caller to specify the list of distinct values to pivot on, and one that does not. The latter is more concise but less efficient, because Spark needs to first compute the list of distinct values internally.

Parameters:

- **pivot_col** – Name of the column to pivot.
- **values** – List of values that will be translated to columns in the output DataFrame.

```
// Compute the sum of earnings for each year by course with each course as a
// separate column >>> df4.groupBy("year").pivot("course", ["dotNET",
// "Java"]).sum("earnings").collect() [Row(year=2012, dotNET=15000, Java=20000),
// Row(year=2013, dotNET=48000, Java=30000)]
```

```
// Or without specifying column values (less efficient) >>>
// df4.groupBy("year").pivot("course").sum("earnings").collect() [Row(year=2012,
// Java=20000, dotNET=15000), Row(year=2013, Java=30000, dotNET=48000)]
```

New in version 1.6.

sum(*args)

Compute the sum for each numeric columns for each group.

Parameters: **cols** – list of column names (string). Non-numeric columns are ignored.

```
>>> df.groupBy().sum('age').collect()
[Row(sum(age)=7)]
>>> df3.groupBy().sum('age', 'height').collect()
[Row(sum(age)=7, sum(height)=165)]
```

New in version 1.3.

class pyspark.sql.Column(jc)

A column in a DataFrame.

Column instances can be created by:

```
# 1. Select a column out of a DataFrame
df.colName
df["colName"]

# 2. Create from an expression
df.colName + 1
1 / df.colName
```

Note: Experimental

New in version 1.3.

alias(*alias)

Returns this column aliased with a new name or names (in the case of expressions that return more than one column, such as explode).

```
>>> df.select(df.age.alias("age2")).collect()
[Row(age2=2), Row(age2=5)]
```

New in version 1.3.

asc()

Returns a sort expression based on the ascending order of the given column name.

astype(dataType)

Convert the column into type dataType.

```
>>> df.select(df.age.cast("string").alias('ages')).collect()
[Row(ages=u'2'), Row(ages=u'5')]
```

```
>>> df.select(df.age.cast(StringType()).alias('ages')).collect()
[Row(ages=u'2'), Row(ages=u'5')]
```

New in version 1.3.

between(lowerBound, upperBound)

A boolean expression that is evaluated to true if the value of this expression is between the given columns.

```
>>> df.select(df.name, df.age.between(2, 4)).show()
+-----+-----+
| name|((age >= 2) && (age <= 4))|
+-----+-----+
| Alice|                                true|
|  Bob|                                false|
+-----+-----+
```

New in version 1.3.

bitwiseAND(other)

binary operator

bitwiseOR(other)

binary operator

bitwiseXOR(other)

binary operator

cast(dataType)

Convert the column into type dataType.

```
>>> df.select(df.age.cast("string").alias('ages')).collect()
[Row(ages=u'2'), Row(ages=u'5')]
>>> df.select(df.age.cast(StringType()).alias('ages')).collect()
[Row(ages=u'2'), Row(ages=u'5')]
```

New in version 1.3.

desc()

Returns a sort expression based on the descending order of the given column name.

endswith(other)

binary operator

getField(name)

An expression that gets a field by name in a StructField.

```
>>> from pyspark.sql import Row
```



```

>>> df = sc.parallelize([Row(r=Row(a=1, b="b"))]).toDF()
>>> df.select(df.r.getField("b")).show()
+----+
|r[b]|
+----+
|   b|
+----+
>>> df.select(df.r.a).show()
+----+
|r[a]|
+----+
|   1|
+----+

```

New in version 1.3.

getItem(key)

An expression that gets an item at position `ordinal` out of a list, or gets an item by key out of a dict.

```

>>> df = sc.parallelize([(1, 2), {"key": "value"}]).toDF(["l", "d"])
>>> df.select(df.l.getItem(0), df.d.getItem("key")).show()
+-----+
|l[0]|d[key]|
+-----+
|   1| value|
+-----+
>>> df.select(df.l[0], df.d["key"]).show()
+-----+
|l[0]|d[key]|
+-----+
|   1| value|
+-----+

```

New in version 1.3.

inSet(*cols)

A boolean expression that is evaluated to true if the value of this expression is contained by the evaluated values of the arguments.

```

>>> df[df.name.inSet("Bob", "Mike")].collect()
[Row(age=5, name=u'Bob')]
>>> df[df.age.inSet(1, 2, 3)].collect()
[Row(age=2, name=u'Alice')]

```

Note: Deprecated in 1.5, use `Column.isin()` instead.

New in version 1.3.

isNotNull()

True if the current expression is not null.

isNull()

True if the current expression is null.

`isin(*cols)`

A boolean expression that is evaluated to true if the value of this expression is contained by the evaluated values of the arguments.

```
>>> df[df.name.isin("Bob", "Mike")].collect()
[Row(age=5, name=u'Bob')]
>>> df[df.age.isin([1, 2, 3])].collect()
[Row(age=2, name=u'Alice')]
```

New in version 1.5.

`like(other)`

binary operator

`otherwise(value)`

Evaluates a list of conditions and returns one of multiple possible result expressions. If `Column.otherwise()` is not invoked, `None` is returned for unmatched conditions.

See `pyspark.sql.functions.when()` for example usage.

Parameters: `value` – a literal value, or a `Column` expression.

```
>>> from pyspark.sql import functions as F
>>> df.select(df.name, F.when(df.age > 3, 1).otherwise(0)).show()
+-----+-----+
| name|CASE WHEN (age > 3) THEN 1 ELSE 0|
+-----+-----+
|Alice|                                0|
|  Bob|                                1|
+-----+-----+
```

New in version 1.4.

`over(window)`

Define a windowing column.

Parameters: `window` – a `WindowSpec`

Returns: a `Column`

```
>>> from pyspark.sql import Window
>>> window = Window.partitionBy("name").orderBy("age").rowsBetween(-1, 1)
>>> from pyspark.sql.functions import rank, min
>>> # df.select(rank().over(window), min('age').over(window))
```

Note: Window functions is only supported with `HiveContext` in 1.4

New in version 1.4.

rlike(*other*)

binary operator

startswith(*other*)

binary operator

substr(*startPos*, *length*)Return a **Column** which is a substring of the column.

Parameters:

- **startPos** – start position (int or **Column**)
- **length** – length of the substring (int or **Column**)

```
>>> df.select(df.name.substr(1, 3).alias("col")).collect()
[Row(col=u'Ali'), Row(col=u'Bob')]
```

*New in version 1.3.***when**(*condition*, *value*)

Evaluates a list of conditions and returns one of multiple possible result expressions. If **Column.otherwise()** is not invoked, **None** is returned for unmatched conditions.

See **pyspark.sql.functions.when()** for example usage.

Parameters:

- **condition** – a boolean **Column** expression.
- **value** – a literal value, or a **Column** expression.

```
>>> from pyspark.sql import functions as F
>>> df.select(df.name, F.when(df.age > 4, 1).when(df.age < 3, -1).otherwise(0)).collect()
+-----+-----+
| name|CASE WHEN (age > 4) THEN 1 WHEN (age < 3) THEN -1 ELSE 0|
+-----+-----+
| Alice| -1|
|  Bob|  1|
+-----+-----+
```

*New in version 1.4.***class** **pyspark.sql.Row**

A row in **DataFrame**. The fields in it can be accessed like attributes.

Row can be used to create a row object by using named arguments, the fields will be sorted by names.

```
>>> row = Row(name="Alice", age=11)
>>> row
Row(age=11, name='Alice')
>>> row['name'], row['age']
('Alice', 11)
>>> row.name, row.age
```

```
('Alice', 11)
```

Row also can be used to create another Row like class, then it could be used to create Row objects, such as

```
>>> Person = Row("name", "age")
>>> Person
<Row(name, age)>
>>> Person("Alice", 11)
Row(name='Alice', age=11)
```

asDict(*recursive=False*)

Return as an dict

Parameters: **recursive** – turns the nested Row as dict (default: False).

```
>>> Row(name="Alice", age=11).asDict() == {'name': 'Alice', 'age': 11}
True
>>> row = Row(key=1, value=Row(name='a', age=2))
>>> row.asDict() == {'key': 1, 'value': Row(age=2, name='a')}
True
>>> row.asDict(True) == {'key': 1, 'value': {'name': 'a', 'age': 2}}
True
```

class pyspark.sql.DataFrameNaFunctions(*df*)

Functionality for working with missing data in **DataFrame**.

New in version 1.4.

drop(*how='any', thresh=None, subset=None*)

Returns a new **DataFrame** omitting rows with null values. **DataFrame.dropna()** and **DataFrameNaFunctions.drop()** are aliases of each other.

Parameters:

- **how** – ‘any’ or ‘all’. If ‘any’, drop a row if it contains any nulls. If ‘all’, drop a row only if all its values are null.
- **thresh** – int, default None. If specified, drop rows that have less than *thresh* non-null values. This overwrites the *how* parameter.
- **subset** – optional list of column names to consider.

```
>>> df4.na.drop().show()
+---+-----+-----+
|age|height| name|
+---+-----+-----+
| 10|    80|Alice|
+---+-----+-----+
```

New in version 1.3.1.

fill(*value, subset=None*)

Replace null values, alias for `na.fill()`. `DataFrame.fillna()` and `DataFrameNaFunctions.fill()` are aliases of each other.

- Parameters:**
- **value** – int, long, float, string, or dict. Value to replace null values with. If the value is a dict, then *subset* is ignored and *value* must be a mapping from column name (string) to replacement value. The replacement value must be an int, long, float, or string.
 - **subset** – optional list of column names to consider. Columns specified in subset that do not have matching data type are ignored. For example, if *value* is a string, and subset contains a non-string column, then the non-string column is simply ignored.

```
>>> df4.na.fill(50).show()
+---+-----+-----+
|age|height|  name|
+---+-----+-----+
| 10|    80|Alice|
|  5|    50|  Bob|
| 50|    50|  Tom|
| 50|    50| null|
+---+-----+-----+
```

```
>>> df4.na.fill({'age': 50, 'name': 'unknown'}).show()
+---+-----+-----+
|age|height|   name|
+---+-----+-----+
| 10|    80|  Alice|
|  5|  null|   Bob|
| 50|  null|   Tom|
| 50|  null|unknown|
+---+-----+-----+
```

New in version 1.3.1.

replace(to_replace, value, subset=None)

Returns a new `DataFrame` replacing a value with another value. `DataFrame.replace()` and `DataFrameNaFunctions.replace()` are aliases of each other.

- Parameters:**
- **to_replace** – int, long, float, string, or list. Value to be replaced. If the value is a dict, then *value* is ignored and *to_replace* must be a mapping from column name (string) to replacement value. The value to be replaced must be an int, long, float, or string.
 - **value** – int, long, float, string, or list. Value to use to replace holes. The replacement value must be an int, long, float, or string. If *value* is a list or tuple, *value* should be of the same length with *to_replace*.
 - **subset** – optional list of column names to consider. Columns specified in subset that do not have matching data type are ignored. For example, if *value* is a string, and subset contains a non-string column, then the non-string column is simply ignored.

```
>>> df4.na.replace(10, 20).show()
+-----+-----+-----+
| age|height| name|
+-----+-----+-----+
|  20|    80|Alice|
|   5|   null|  Bob|
| null|   null|  Tom|
| null|   null| null|
+-----+-----+-----+
```

```
>>> df4.na.replace(['Alice', 'Bob'], ['A', 'B'], 'name').show()
+-----+-----+-----+
| age|height|name|
+-----+-----+-----+
|  10|    80|  A|
|   5|   null|  B|
| null|   null|Tom|
| null|   null|null|
+-----+-----+-----+
```

New in version 1.4.

`class pyspark.sql.DataFrameStatFunctions(df)`

Functionality for statistic functions with `DataFrame`.

New in version 1.4.

`corr(col1, col2, method=None)`

Calculates the correlation of two columns of a `DataFrame` as a double value. Currently only supports the Pearson Correlation Coefficient. `DataFrame.corr()` and `DataFrameStatFunctions.corr()` are aliases of each other.

Parameters:

- **col1** – The name of the first column
- **col2** – The name of the second column
- **method** – The correlation method. Currently only supports “pearson”

New in version 1.4.

`cov(col1, col2)`

Calculate the sample covariance for the given columns, specified by their names, as a double value. `DataFrame.cov()` and `DataFrameStatFunctions.cov()` are aliases.

Parameters:

- **col1** – The name of the first column
- **col2** – The name of the second column

New in version 1.4.

`crosstab(col1, col2)`

Computes a pair-wise frequency table of the given columns. Also known as a contingency table. The number of distinct values for each column should be less than

1e4. At most 1e6 non-zero pair frequencies will be returned. The first column of each row will be the distinct values of *col1* and the column names will be the distinct values of *col2*. The name of the first column will be *\$col1_\$col2*. Pairs that have no occurrences will have zero as their counts. `DataFrame.crosstab()` and `DataFrameStatFunctions.crosstab()` are aliases.

Parameters:

- **col1** – The name of the first column. Distinct items will make the first item of each row.
- **col2** – The name of the second column. Distinct items will make the column names of the DataFrame.

New in version 1.4.

`freqItems(cols, support=None)`

Finding frequent items for columns, possibly with false positives. Using the frequent element count algorithm described in “<http://dx.doi.org/10.1145/762471.762473>, proposed by Karp, Schenker, and Papadimitriou”. `DataFrame.freqItems()` and `DataFrameStatFunctions.freqItems()` are aliases.

Note: This function is meant for exploratory data analysis, as we make no guarantee about the backward compatibility of the schema of the resulting DataFrame.

Parameters:

- **cols** – Names of the columns to calculate frequent items for as a list or tuple of strings.
- **support** – The frequency with which to consider an item ‘frequent’. Default is 1%. The support must be greater than 1e-4.

New in version 1.4.

`sampleBy(col, fractions, seed=None)`

Returns a stratified sample without replacement based on the fraction given on each stratum.

Parameters:

- **col** – column that defines strata
- **fractions** – sampling fraction for each stratum. If a stratum is not specified, we treat its fraction as zero.
- **seed** – random seed

Returns: a new DataFrame that represents the stratified sample

```
>>> from pyspark.sql.functions import col
>>> dataset = sqlContext.range(0, 100).select((col("id") % 3).alias("key"))
>>> sampled = dataset.sampleBy("key", fractions={0: 0.1, 1: 0.2}, seed=0)
>>> sampled.groupBy("key").count().orderBy("key").show()
+----+-----+
|key|count|
+----+-----+
|  0|     5|
|  1|     9|
+----+-----+
```

New in version 1.5.

`class pyspark.sql.Window`

Utility functions for defining window in DataFrames.

For example:

```
>>> # PARTITION BY country ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CUR
>>> window = Window.partitionBy("country").orderBy("date").rowsBetween(-sys.maxsi
```

```
>>> # PARTITION BY country ORDER BY date RANGE BETWEEN 3 PRECEDING AND 3 FOLLOWIN
>>> window = Window.orderBy("date").partitionBy("country").rangeBetween(-3, 3)
```

Note: Experimental

New in version 1.4.

`static orderBy(*cols)`

Creates a `WindowSpec` with the partitioning defined.

New in version 1.4.

`static partitionBy(*cols)`

Creates a `WindowSpec` with the partitioning defined.

New in version 1.4.

`class pyspark.sql.WindowSpec(jspec)`

A window specification that defines the partitioning, ordering, and frame boundaries.

Use the static methods in `Window` to create a `WindowSpec`.

Note: Experimental

New in version 1.4.

`orderBy(*cols)`

Defines the ordering columns in a `WindowSpec`.

Parameters: `cols` – names of columns or expressions

New in version 1.4.

`partitionBy(*cols)`

Defines the partitioning columns in a `WindowSpec`.

Parameters: **cols** – names of columns or expressions

New in version 1.4.

rangeBetween(*start*, *end*)

Defines the frame boundaries, from *start* (inclusive) to *end* (inclusive).

Both *start* and *end* are relative from the current row. For example, “0” means “current row”, while “-1” means one off before the current row, and “5” means the five off after the current row.

Parameters:

- **start** – boundary start, inclusive. The frame is unbounded if this is `-sys.maxsize` (or lower).
- **end** – boundary end, inclusive. The frame is unbounded if this is `sys.maxsize` (or higher).

New in version 1.4.

rowsBetween(*start*, *end*)

Defines the frame boundaries, from *start* (inclusive) to *end* (inclusive).

Both *start* and *end* are relative positions from the current row. For example, “0” means “current row”, while “-1” means the row before the current row, and “5” means the fifth row after the current row.

Parameters:

- **start** – boundary start, inclusive. The frame is unbounded if this is `-sys.maxsize` (or lower).
- **end** – boundary end, inclusive. The frame is unbounded if this is `sys.maxsize` (or higher).

New in version 1.4.

class `pyspark.sql.DataFrameReader`(*sqlContext*)

Interface used to load a **DataFrame** from external storage systems (e.g. file systems, key-value stores, etc). Use **SQLContext.read()** to access this.

::Note: Experimental

New in version 1.4.

format(*source*)

Specifies the input data source format.

Parameters: **source** – string, name of the data source, e.g. ‘json’, ‘parquet’.

```
>>> df = sqlContext.read.format('json').load('python/test_support/sql/people')
>>> df.dtypes
[('age', 'bigint'), ('name', 'string')]
```

New in version 1.4.

jdbc(*url*, *table*, *column*=None, *lowerBound*=None, *upperBound*=None, *numPartitions*=None, *predicates*=None, *properties*=None)

Construct a **DataFrame** representing the database table accessible via JDBC URL *url* named *table* and connection *properties*.

The *column* parameter could be used to partition the table, then it will be retrieved in parallel based on the parameters passed to this function.

The *predicates* parameter gives a list expressions suitable for inclusion in WHERE clauses; each one defines one partition of the **DataFrame**.

::Note: Don't create too many partitions in parallel on a large cluster; otherwise Spark might crash your external database systems.

Parameters:

- **url** – a JDBC URL
- **table** – name of table
- **column** – the column used to partition
- **lowerBound** – the lower bound of partition column
- **upperBound** – the upper bound of the partition column
- **numPartitions** – the number of partitions
- **predicates** – a list of expressions
- **properties** – JDBC database connection arguments, a list of arbitrary string tag/value. Normally at least a “user” and “password” property should be included.

Returns: a DataFrame

New in version 1.4.

json(*path*, *schema*=None)

Loads a JSON file (one object per line) or an RDD of Strings storing JSON objects (one object per record) and returns the result as a :class`DataFrame`.

If the *schema* parameter is not specified, this function goes through the input once to determine the input schema.

Parameters:

- **path** – string represents path to the JSON dataset, or RDD of Strings storing JSON objects.
- **schema** – an optional **StructType** for the input schema.

You can set the following JSON-specific options to deal with non-standard JSON files:

- **primitivesAsString** (default false): infers all primitive values as a string type
- **allowComments** (default false): ignores Java/C++ style comment in JSON records
- **allowUnquotedFieldNames** (default false): allows unquoted JSON field names
- **allowSingleQuotes** (default true): allows single quotes in addition to double

quotes

- `allowNumericLeadingZeros` (default `false`): allows leading zeros in numbers (e.g. 00012)

```
>>> df1 = sqlContext.read.json('python/test_support/sql/people.json')
>>> df1.dtypes
[('age', 'bigint'), ('name', 'string')]
>>> rdd = sc.textFile('python/test_support/sql/people.json')
>>> df2 = sqlContext.read.json(rdd)
>>> df2.dtypes
[('age', 'bigint'), ('name', 'string')]
```

New in version 1.4.

load(*path=None, format=None, schema=None, **options*)

Loads data from a data source and returns it as a :class:`DataFrame`.

- Parameters:**
- **path** – optional string or a list of string for file-system backed data sources.
 - **format** – optional string for format of the data source. Default to 'parquet'.
 - **schema** – optional **StructType** for the input schema.
 - **options** – all other string options

```
>>> df = sqlContext.read.load('python/test_support/sql/parquet_partitioned',
...     opt2=1, opt3='str')
>>> df.dtypes
[('name', 'string'), ('year', 'int'), ('month', 'int'), ('day', 'int')]
```

```
>>> df = sqlContext.read.format('json').load(['python/test_support/sql/people',
...     'python/test_support/sql/people1.json'])
>>> df.dtypes
[('age', 'bigint'), ('aka', 'string'), ('name', 'string')]
```

New in version 1.4.

option(*key, value*)

Adds an input option for the underlying data source.

New in version 1.5.

options(***options*)

Adds input options for the underlying data source.

New in version 1.4.

orc(*path*)

Loads an ORC file, returning the result as a **DataFrame**.

::Note: Currently ORC support is only available together with **HiveContext**.

```
>>> df = hiveContext.read.orc('python/test_support/sql/orc_partitioned')
>>> df.dtypes
[('a', 'bigint'), ('b', 'int'), ('c', 'int')]
```

New in version 1.5.

parquet(*paths)

Loads a Parquet file, returning the result as a **DataFrame**.

```
>>> df = sqlContext.read.parquet('python/test_support/sql/parquet_partitioned')
>>> df.dtypes
[('name', 'string'), ('year', 'int'), ('month', 'int'), ('day', 'int')]
```

New in version 1.4.

schema(schema)

Specifies the input schema.

Some data sources (e.g. JSON) can infer the input schema automatically from data. By specifying the schema here, the underlying data source can skip the schema inference step, and thus speed up data loading.

Parameters: **schema** – a StructType object

New in version 1.4.

table(tableName)

Returns the specified table as a **DataFrame**.

Parameters: **tableName** – string, name of the table.

```
>>> df = sqlContext.read.parquet('python/test_support/sql/parquet_partitioned')
>>> df.registerTempTable('tmpTable')
>>> sqlContext.read.table('tmpTable').dtypes
[('name', 'string'), ('year', 'int'), ('month', 'int'), ('day', 'int')]
```

New in version 1.4.

text(paths)

Loads a text file and returns a **DataFrame** with a single string column named “value”.

Each line in the text file is a new row in the resulting **DataFrame**.

Parameters: **paths** – string, or list of strings, for input path(s).

```
>>> df = sqlContext.read.text('python/test_support/sql/text-test.txt')
>>> df.collect()
[Row(value=u'hello'), Row(value=u'this')]
```

New in version 1.6.

class pyspark.sql.DataFrameWriter(*df*)

Interface used to write a [[DataFrame]] to external storage systems (e.g. file systems, key-value stores, etc). Use **DataFrame.write()** to access this.

::Note: Experimental

New in version 1.4.

format(*source*)

Specifies the underlying output data source.

Parameters: **source** – string, name of the data source, e.g. 'json', 'parquet'.

```
>>> df.write.format('json').save(os.path.join(tempfile.mkdtemp(), 'data'))
```

New in version 1.4.

insertInto(*tableName, overwrite=False*)

Inserts the content of the **DataFrame** to the specified table.

It requires that the schema of the class:*DataFrame* is the same as the schema of the table.

Optionally overwriting any existing data.

New in version 1.4.

jdbc(*url, table, mode=None, properties=None*)

Saves the content of the **DataFrame** to a external database table via JDBC.

Note: Don't create too many partitions in parallel on a large cluster; otherwise Spark might crash your external database systems.

Parameters:

- **url** – a JDBC URL of the form `jdbc:subprotocol:subname`
- **table** – Name of the table in the external database.
- **mode** – specifies the behavior of the save operation when data already exists.
 - **append**: Append contents of this **DataFrame** to existing data.
 - **overwrite**: Overwrite existing data.

- **ignore**: Silently ignore this operation if data already exists.
- **error** (default case): Throw an exception if data already exists.
- **properties** – JDBC database connection arguments, a list of arbitrary string tag/value. Normally at least a “user” and “password” property should be included.

New in version 1.4.

json(path, mode=None)

Saves the content of the **DataFrame** in JSON format at the specified path.

- Parameters:**
- **path** – the path in any Hadoop supported file system
 - **mode** – specifies the behavior of the save operation when data already exists.
 - **append**: Append contents of this **DataFrame** to existing data.
 - **overwrite**: Overwrite existing data.
 - **ignore**: Silently ignore this operation if data already exists.
 - **error** (default case): Throw an exception if data already exists.

```
>>> df.write.json(os.path.join(tempfile.mkdtemp(), 'data'))
```

New in version 1.4.

mode(saveMode)

Specifies the behavior when data or table already exists.

Options include:

- **append**: Append contents of this **DataFrame** to existing data.
- **overwrite**: Overwrite existing data.
- **error**: Throw an exception if data already exists.
- **ignore**: Silently ignore this operation if data already exists.

```
>>> df.write.mode('append').parquet(os.path.join(tempfile.mkdtemp(), 'data'))
```

New in version 1.4.

option(key, value)

Adds an output option for the underlying data source.

New in version 1.5.

options(**options)

Adds output options for the underlying data source.

New in version 1.4.

orc(path, mode=None, partitionBy=None)

Saves the content of the **DataFrame** in ORC format at the specified path.

::Note: Currently ORC support is only available together with **HiveContext**.

- Parameters:**
- **path** – the path in any Hadoop supported file system
 - **mode** – specifies the behavior of the save operation when data already exists.
 - **append**: Append contents of this **DataFrame** to existing data.
 - **overwrite**: Overwrite existing data.
 - **ignore**: Silently ignore this operation if data already exists.
 - **error** (default case): Throw an exception if data already exists.
 - **partitionBy** – names of partitioning columns

```
>>> orc_df = hiveContext.read.orc('python/test_support/sql/orc_partitioned')
>>> orc_df.write.orc(os.path.join(tempfile.mkdtemp(), 'data'))
```

New in version 1.5.

parquet(path, mode=None, partitionBy=None)

Saves the content of the **DataFrame** in Parquet format at the specified path.

- Parameters:**
- **path** – the path in any Hadoop supported file system
 - **mode** – specifies the behavior of the save operation when data already exists.
 - **append**: Append contents of this **DataFrame** to existing data.
 - **overwrite**: Overwrite existing data.
 - **ignore**: Silently ignore this operation if data already exists.
 - **error** (default case): Throw an exception if data already exists.
 - **partitionBy** – names of partitioning columns

```
>>> df.write.parquet(os.path.join(tempfile.mkdtemp(), 'data'))
```

New in version 1.4.

partitionBy(*cols)

Partitions the output by the given columns on the file system.

If specified, the output is laid out on the file system similar to Hive's partitioning scheme.

- Parameters:**
- **cols** – name of columns

```
>>> df.write.partitionBy('year', 'month').parquet(os.path.join(tempfile.mkdtemp(), 'data'))
```

New in version 1.4.

save(*path=None, format=None, mode=None, partitionBy=None, **options*)

Saves the contents of the **DataFrame** to a data source.

The data source is specified by the format and a set of options. If format is not specified, the default data source configured by `spark.sql.sources.default` will be used.

- Parameters:**
- **path** – the path in a Hadoop supported file system
 - **format** – the format used to save
 - **mode** – specifies the behavior of the save operation when data already exists.
 - `append`: Append contents of this **DataFrame** to existing data.
 - `overwrite`: Overwrite existing data.
 - `ignore`: Silently ignore this operation if data already exists.
 - `error` (default case): Throw an exception if data already exists.
 - **partitionBy** – names of partitioning columns
 - **options** – all other string options

```
>>> df.write.mode('append').parquet(os.path.join(tempfile.mkdtemp(), 'data'))
```

New in version 1.4.

saveAsTable(*name, format=None, mode=None, partitionBy=None, **options*)

Saves the content of the **DataFrame** as the specified table.

In the case the table already exists, behavior of this function depends on the save mode, specified by the *mode* function (default to throwing an exception). When *mode* is *Overwrite*, the schema of the `[[DataFrame]]` does not need to be the same as that of the existing table.

- *append*: Append contents of this **DataFrame** to existing data.
- *overwrite*: Overwrite existing data.
- *error*: Throw an exception if data already exists.
- *ignore*: Silently ignore this operation if data already exists.

- Parameters:**
- **name** – the table name
 - **format** – the format used to save
 - **mode** – one of *append*, *overwrite*, *error*, *ignore* (default: *error*)
 - **partitionBy** – names of partitioning columns
 - **options** – all other string options

New in version 1.4.

text(*path*)

Saves the content of the DataFrame in a text file at the specified path.

The DataFrame must have only one column that is of string type. Each row becomes a new line in the output file.

New in version 1.6.

pyspark.sql.types module

class pyspark.sql.types.**DataType** [\[source\]](#)

Base class for data types.

fromInternal(*obj*) [\[source\]](#)

Converts an internal SQL object into a native Python object.

json() [\[source\]](#)

jsonValue() [\[source\]](#)

needConversion() [\[source\]](#)

Does this type need to conversion between Python object and internal SQL object.

This is used to avoid the unnecessary conversion for ArrayType/MapType/StructType.

simpleString() [\[source\]](#)

toInternal(*obj*) [\[source\]](#)

Converts a Python object into an internal SQL object.

classmethod **typeName**() [\[source\]](#)

class pyspark.sql.types.**NullType** [\[source\]](#)

Null type.

The data type representing None, used for the types that cannot be inferred.

class pyspark.sql.types.**StringType** [\[source\]](#)

String data type.

class pyspark.sql.types.**BinaryType** [\[source\]](#)

Binary (byte array) data type.

class pyspark.sql.types.**BooleanType** [\[source\]](#)

Boolean data type.

class pyspark.sql.types.**DateType** [\[source\]](#)

Date (datetime.date) data type.

EPOCH_ORDINAL = 719163

fromInternal(*v*) [\[source\]](#)

needConversion() [\[source\]](#)

toInternal(*d*) [\[source\]](#)

class pyspark.sql.types.**TimestampType** [\[source\]](#)

Timestamp (datetime.datetime) data type.

fromInternal(*ts*) [\[source\]](#)

needConversion() [\[source\]](#)

toInternal(*dt*) [\[source\]](#)

class pyspark.sql.types.**DecimalType**(*precision=10, scale=0*) [\[source\]](#)

Decimal (decimal.Decimal) data type.

The DecimalType must have fixed precision (the maximum total number of digits) and scale (the number of digits on the right of dot). For example, (5, 2) can support the value from [-999.99 to 999.99].

The precision can be up to 38, the scale must less or equal to precision.

When create a DecimalType, the default precision and scale is (10, 0). When infer schema from decimal.Decimal objects, it will be DecimalType(38, 18).

Parameters:

- **precision** – the maximum total number of digits (default: 10)
- **scale** – the number of digits on right side of dot. (default: 0)

jsonValue() [\[source\]](#)

simpleString() [\[source\]](#)

class pyspark.sql.types.**DoubleType** [\[source\]](#)

Double data type, representing double precision floats.

class pyspark.sql.types.**FloatType** [\[source\]](#)

Float data type, representing single precision floats.

class pyspark.sql.types.**ByteType** [\[source\]](#)

Byte data type, i.e. a signed integer in a single byte.

simpleString() [\[source\]](#)

class pyspark.sql.types.**IntegerType** [\[source\]](#)

Int data type, i.e. a signed 32-bit integer.

simpleString() [\[source\]](#)

class pyspark.sql.types.**LongType** [\[source\]](#)

Long data type, i.e. a signed 64-bit integer.

If the values are beyond the range of [-9223372036854775808, 9223372036854775807], please use **DecimalType**.

simpleString() [\[source\]](#)

class pyspark.sql.types.**ShortType** [\[source\]](#)

Short data type, i.e. a signed 16-bit integer.

simpleString() [\[source\]](#)

class pyspark.sql.types.**ArrayType**(*elementType*, *containsNull=True*) [\[source\]](#)

Array data type.

Parameters:

- **elementType** – **DataType** of each element in the array.
- **containsNull** – boolean, whether the array can contain null (None) values.

fromInternal(obj) [\[source\]](#)

classmethod **fromJson(json)** [\[source\]](#)

jsonValue() [\[source\]](#)

needConversion() [\[source\]](#)

simpleString() [\[source\]](#)

toInternal(obj) [\[source\]](#)

class pyspark.sql.types.**MapType**(*keyType*, *valueType*, *valueContainsNull=True*) [\[source\]](#)

Map data type.

Parameters:

- **keyType** – **DataType** of the keys in the map.
- **valueType** – **DataType** of the values in the map.
- **valueContainsNull** – indicates whether values can contain null (None) values.

Keys in a map data type are not allowed to be null (None).

fromInternal(obj) [\[source\]](#)

classmethod **fromJson(json)** [\[source\]](#)

jsonValue() [\[source\]](#)

needConversion() [\[source\]](#)

`simpleString()` [\[source\]](#)

`toInternal(obj)` [\[source\]](#)

`class pyspark.sql.types.StructField(name, dataType, nullable=True, metadata=None)` [\[source\]](#)
 A field in `StructType`.

Parameters:

- **name** – string, name of the field.
- **dataType** – `DataType` of the field.
- **nullable** – boolean, whether the field can be null (`None`) or not.
- **metadata** – a dict from string to simple type that can be `toInternal`d to JSON automatically

`fromInternal(obj)` [\[source\]](#)

`classmethod fromJson(json)` [\[source\]](#)

`jsonValue()` [\[source\]](#)

`needConversion()` [\[source\]](#)

`simpleString()` [\[source\]](#)

`toInternal(obj)` [\[source\]](#)

`class pyspark.sql.types.StructType(fields=None)` [\[source\]](#)
 Struct type, consisting of a list of `StructField`.

This is the data type representing a `Row`.

`add(field, data_type=None, nullable=True, metadata=None)` [\[source\]](#)

Construct a `StructType` by adding new elements to it to define the schema. The method accepts either:

- A single parameter which is a `StructField` object.
- Between 2 and 4 parameters as (name, data_type, nullable (optional), metadata(optional)). The data_type parameter may be either a `String` or a `DataType` object.

```
>>> struct1 = StructType().add("f1", StringType(), True).add("f2", StringType(), True)
>>> struct2 = StructType([StructField("f1", StringType(), True), StructField("f2", StringType(), True)])
>>> struct1 == struct2
True
>>> struct1 = StructType().add(StructField("f1", StringType(), True))
>>> struct2 = StructType([StructField("f1", StringType(), True)])
>>> struct1 == struct2
True
>>> struct1 = StructType().add("f1", "string", True)
>>> struct2 = StructType([StructField("f1", StringType(), True)])
>>> struct1 == struct2
True
```

Parameters:

- **field** – Either the name of the field or a StructField object
- **data_type** – If present, the DataType of the StructField to create
- **nullable** – Whether the field to add should be nullable (default True)
- **metadata** – Any additional metadata (default None)

Returns: a new updated StructType

`fromInternal(obj)` [\[source\]](#)

`classmethod fromJson(json)` [\[source\]](#)

`jsonValue()` [\[source\]](#)

`needConversion()` [\[source\]](#)

`simpleString()` [\[source\]](#)

`toInternal(obj)` [\[source\]](#)

pyspark.sql.functions module

A collections of builtin functions

`pyspark.sql.functions.abs(col)`
Computes the absolute value.

New in version 1.3.

`pyspark.sql.functions.acos(col)`
Computes the cosine inverse of the given value; the returned angle is in the range 0.0 through pi.

New in version 1.4.

`pyspark.sql.functions.add_months(start, months)` [\[source\]](#)
Returns the date that is *months* months after *start*

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['d'])
>>> df.select(add_months(df.d, 1).alias('d')).collect()
[Row(d=datetime.date(2015, 5, 8))]
```

New in version 1.5.

`pyspark.sql.functions.approxCountDistinct(col, rsd=None)` [\[source\]](#)
Returns a new **Column** for approximate distinct count of *col*.

```
>>> df.agg(approxCountDistinct(df.age).alias('c')).collect()
[Row(c=2)]
```

New in version 1.3.

`pyspark.sql.functions.array(*cols)`

[\[source\]](#)

Creates a new array column.

Parameters: **cols** – list of column names (string) or list of `Column` expressions that have the same data type.

```
>>> df.select(array('age', 'age').alias("arr")).collect()
[Row(arr=[2, 2]), Row(arr=[5, 5])]
>>> df.select(array([df.age, df.age]).alias("arr")).collect()
[Row(arr=[2, 2]), Row(arr=[5, 5])]
```

New in version 1.4.

`pyspark.sql.functions.array_contains(col, value)`

[\[source\]](#)

Collection function: returns True if the array contains the given value. The collection elements and value must be of the same type.

Parameters:

- **col** – name of column containing array
- **value** – value to check for in array

```
>>> df = sqlContext.createDataFrame([(["a", "b", "c"],), ([],)], ['data'])
>>> df.select(array_contains(df.data, "a")).collect()
[Row(array_contains(data,a)=True), Row(array_contains(data,a)=False)]
```

New in version 1.5.

`pyspark.sql.functions.asc(col)`

Returns a sort expression based on the ascending order of the given column name.

New in version 1.3.

`pyspark.sql.functions.ascii(col)`

Computes the numeric value of the first character of the string column.

New in version 1.5.

`pyspark.sql.functions.asin(col)`

Computes the sine inverse of the given value; the returned angle is in the range- $\pi/2$ through $\pi/2$.

New in version 1.4.

`pyspark.sql.functions.atan(col)`

Computes the tangent inverse of the given value.

New in version 1.4.

`pyspark.sql.functions.atan2(col1, col2)`

Returns the angle theta from the conversion of rectangular coordinates (x, y) to polar coordinates (r, theta).

New in version 1.4.

`pyspark.sql.functions.avg(col)`

Aggregate function: returns the average of the values in a group.

New in version 1.3.

`pyspark.sql.functions.base64(col)`

Computes the BASE64 encoding of a binary column and returns it as a string column.

New in version 1.5.

`pyspark.sql.functions.bin(col)`

[\[source\]](#)

Returns the string representation of the binary value of the given column.

```
>>> df.select(bin(df.age).alias('c')).collect()
[Row(c=u'10'), Row(c=u'101')]
```

New in version 1.5.

`pyspark.sql.functions.bitwiseNOT(col)`

Computes bitwise not.

New in version 1.4.

`pyspark.sql.functions.broadcast(df)`

[\[source\]](#)

Marks a DataFrame as small enough for use in broadcast joins.

New in version 1.6.

`pyspark.sql.functions.cbrt(col)`

Computes the cube-root of the given value.

New in version 1.4.

`pyspark.sql.functions.ceil(col)`

Computes the ceiling of the given value.

New in version 1.4.

`pyspark.sql.functions.coalesce(*cols)`

[\[source\]](#)

Returns the first column that is not null.

```
>>> cDf = sqlContext.createDataFrame([(None, None), (1, None), (None, 2)], ("a",
>>> cDf.show()
+----+----+
|  a |  b |
+----+----+
| null | null |
|  1 | null |
| null |  2 |
+----+----+
```

```
>>> cDf.select(coalesce(cDf["a"], cDf["b"])).show()
+-----+
|coalesce(a,b)|
+-----+
|          null|
|             1|
|             2|
+-----+
```

```
>>> cDf.select('*', coalesce(cDf["a"], lit(0.0))).show()
+----+----+-----+
|  a |  b |coalesce(a,0.0)|
+----+----+-----+
| null | null |          0.0 |
|  1 | null |          1.0 |
| null |  2 |          0.0 |
+----+----+-----+
```

New in version 1.4.

`pyspark.sql.functions.col(col)`

Returns a **Column** based on the given column name.

New in version 1.3.

`pyspark.sql.functions.collect_list(col)`

Aggregate function: returns a list of objects with duplicates.

New in version 1.6.

`pyspark.sql.functions.collect_set(col)`

Aggregate function: returns a set of objects with duplicate elements eliminated.

New in version 1.6.

`pyspark.sql.functions.column(col)`

Returns a **Column** based on the given column name.

New in version 1.3.

`pyspark.sql.functions.concat(*cols)`

[\[source\]](#)

Concatenates multiple input string columns together into a single string column.

```
>>> df = sqlContext.createDataFrame([('abcd', '123')], ['s', 'd'])
>>> df.select(concat(df.s, df.d).alias('s')).collect()
[Row(s=u'abcd123')]
```

New in version 1.5.

pyspark.sql.functions.**concat_ws**(sep, *cols)

[\[source\]](#)

Concatenates multiple input string columns together into a single string column, using the given separator.

```
>>> df = sqlContext.createDataFrame([('abcd', '123')], ['s', 'd'])
>>> df.select(concat_ws('-', df.s, df.d).alias('s')).collect()
[Row(s=u'abcd-123')]
```

New in version 1.5.

pyspark.sql.functions.**conv**(col, fromBase, toBase)

[\[source\]](#)

Convert a number in a string column from one base to another.

```
>>> df = sqlContext.createDataFrame(["010101",], ['n'])
>>> df.select(conv(df.n, 2, 16).alias('hex')).collect()
[Row(hex=u'15')]
```

New in version 1.5.

pyspark.sql.functions.**corr**(col1, col2)

[\[source\]](#)

Returns a new **Column** for the Pearson Correlation Coefficient for col1 and col2.

```
>>> a = [x * x - 2 * x + 3.5 for x in range(20)]
>>> b = range(20)
>>> corrDf = sqlContext.createDataFrame(zip(a, b))
>>> corrDf = corrDf.agg(corr(corrDf._1, corrDf._2).alias('c'))
>>> corrDf.selectExpr('abs(c - 0.9572339139475857) < 1e-16 as t').collect()
[Row(t=True)]
```

New in version 1.6.

pyspark.sql.functions.**cos**(col)

Computes the cosine of the given value.

New in version 1.4.

pyspark.sql.functions.**cosh**(col)

Computes the hyperbolic cosine of the given value.

New in version 1.4.

`pyspark.sql.functions.count(col)`

Aggregate function: returns the number of items in a group.

New in version 1.3.

`pyspark.sql.functions.countDistinct(col, *cols)`

[\[source\]](#)

Returns a new **Column** for distinct count of `col` or `cols`.

```
>>> df.agg(countDistinct(df.age, df.name).alias('c')).collect()
[Row(c=2)]
```

```
>>> df.agg(countDistinct("age", "name").alias('c')).collect()
[Row(c=2)]
```

New in version 1.3.

`pyspark.sql.functions.crc32(col)`

[\[source\]](#)

Calculates the cyclic redundancy check value (CRC32) of a binary column and returns the value as a bigint.

```
>>> sqlContext.createDataFrame([('ABC',)], ['a']).select(crc32('a').alias('crc32'))
[Row(crc32=2743272264)]
```

New in version 1.5.

`pyspark.sql.functions.cumeDist()`

Window function: .. note:: Deprecated in 1.6, use `cume_dist` instead.

New in version 1.6.

`pyspark.sql.functions.cume_dist()`

Window function: returns the cumulative distribution of values within a window partition, i.e. the fraction of rows that are below the current row.

New in version 1.6.

`pyspark.sql.functions.current_date()`

[\[source\]](#)

Returns the current date as a date column.

New in version 1.5.

`pyspark.sql.functions.current_timestamp()`

[\[source\]](#)

Returns the current timestamp as a timestamp column.

`pyspark.sql.functions.date_add(start, days)`

[\[source\]](#)

Returns the date that is *days* days after *start*

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['d'])
>>> df.select(date_add(df.d, 1).alias('d')).collect()
[Row(d=datetime.date(2015, 4, 9))]
```

New in version 1.5.

`pyspark.sql.functions.date_format(date, format)`

[\[source\]](#)

Converts a date/timestamp/string to a value of string in the format specified by the date format given by the second argument.

A pattern could be for instance *dd.MM.yyyy* and could return a string like '18.03.1993'. All pattern letters of the Java class *java.text.SimpleDateFormat* can be used.

NOTE: Use when ever possible specialized functions like *year*. These benefit from a specialized implementation.

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['a'])
>>> df.select(date_format('a', 'MM/dd/yyyy').alias('date')).collect()
[Row(date=u'04/08/2015')]
```

New in version 1.5.

`pyspark.sql.functions.date_sub(start, days)`

[\[source\]](#)

Returns the date that is *days* days before *start*

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['d'])
>>> df.select(date_sub(df.d, 1).alias('d')).collect()
[Row(d=datetime.date(2015, 4, 7))]
```

New in version 1.5.

`pyspark.sql.functions.datediff(end, start)`

[\[source\]](#)

Returns the number of days from *start* to *end*.

```
>>> df = sqlContext.createDataFrame([('2015-04-08', '2015-05-10')], ['d1', 'd2'])
>>> df.select(datediff(df.d2, df.d1).alias('diff')).collect()
[Row(diff=32)]
```

New in version 1.5.

`pyspark.sql.functions.dayofmonth(col)`

[\[source\]](#)

Extract the day of the month of a given date as integer.

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['a'])
>>> df.select(dayofmonth('a').alias('day')).collect()
[Row(day=8)]
```

New in version 1.5.

`pyspark.sql.functions.dayofyear(col)`

[\[source\]](#)

Extract the day of the year of a given date as integer.

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['a'])
>>> df.select(dayofyear('a').alias('day')).collect()
[Row(day=98)]
```

New in version 1.5.

`pyspark.sql.functions.decode(col, charset)`

[\[source\]](#)

Computes the first argument into a string from a binary using the provided character set (one of 'US-ASCII', 'ISO-8859-1', 'UTF-8', 'UTF-16BE', 'UTF-16LE', 'UTF-16').

New in version 1.5.

`pyspark.sql.functions.denseRank()`

Window function: .. note:: Deprecated in 1.6, use `dense_rank` instead.

New in version 1.6.

`pyspark.sql.functions.dense_rank()`

Window function: returns the rank of rows within a window partition, without any gaps.

The difference between `rank` and `denseRank` is that `denseRank` leaves no gaps in ranking sequence when there are ties. That is, if you were ranking a competition using `denseRank` and had three people tie for second place, you would say that all three were in second place and that the next person came in third.

New in version 1.6.

`pyspark.sql.functions.desc(col)`

Returns a sort expression based on the descending order of the given column name.

New in version 1.3.

`pyspark.sql.functions.encode(col, charset)`

[\[source\]](#)

Computes the first argument into a binary from a string using the provided character set (one of 'US-ASCII', 'ISO-8859-1', 'UTF-8', 'UTF-16BE', 'UTF-16LE', 'UTF-16').

New in version 1.5.

`pyspark.sql.functions.exp(col)`

Computes the exponential of the given value.

New in version 1.4.

`pyspark.sql.functions.explode(col)`

[\[source\]](#)

Returns a new row for each element in the given array or map.

```
>>> from pyspark.sql import Row
>>> eDF = sqlContext.createDataFrame([Row(a=1, intlist=[1,2,3], mapfield={"a": "b
>>> eDF.select(explode(eDF.intlist).alias("anInt")).collect()
[Row(anInt=1), Row(anInt=2), Row(anInt=3)]
```

```
>>> eDF.select(explode(eDF.mapfield).alias("key", "value")).show()
+---+-----+
|key|value|
+---+-----+
|  a|    b|
+---+-----+
```

New in version 1.4.

`pyspark.sql.functions.expm1(col)`

Computes the exponential of the given value minus one.

New in version 1.4.

`pyspark.sql.functions.expr(str)`

[\[source\]](#)

Parses the expression string into the column that it represents

```
>>> df.select(expr("length(name)").collect()
[Row(length(name)=5), Row(length(name)=3)]
```

New in version 1.5.

`pyspark.sql.functions.factorial(col)`

[\[source\]](#)

Computes the factorial of the given value.

```
>>> df = sqlContext.createDataFrame([(5,)], ['n'])
>>> df.select(factorial(df.n).alias('f')).collect()
[Row(f=120)]
```

New in version 1.5.

`pyspark.sql.functions.first(col)`

Aggregate function: returns the first value in a group.

New in version 1.3.

`pyspark.sql.functions.floor(col)`

Computes the floor of the given value.

New in version 1.4.

`pyspark.sql.functions.format_number(col, d)` [\[source\]](#)

Formats the number X to a format like '#,-#,-#.-', rounded to d decimal places, and returns the result as a string.

Parameters:

- **col** – the column name of the numeric value to be formatted
- **d** – the N decimal places

```
>>> sqlContext.createDataFrame([(5,)], ['a']).select(format_number('a', 4).alias([Row(v=u'5.0000')])
```

New in version 1.5.

`pyspark.sql.functions.format_string(format, *cols)` [\[source\]](#)

Formats the arguments in printf-style and returns the result as a string column.

Parameters:

- **col** – the column name of the numeric value to be formatted
- **d** – the N decimal places

```
>>> df = sqlContext.createDataFrame([(5, "hello")], ['a', 'b'])
>>> df.select(format_string('%d %s', df.a, df.b).alias('v')).collect()
[Row(v=u'5 hello')]
```

New in version 1.5.

`pyspark.sql.functions.from_unixtime(timestamp, format='yyyy-MM-dd HH:mm:ss')` [\[source\]](#)

Converts the number of seconds from unix epoch (1970-01-01 00:00:00 UTC) to a string representing the timestamp of that moment in the current system time zone in the given format.

New in version 1.5.

`pyspark.sql.functions.from_utc_timestamp(timestamp, tz)` [\[source\]](#)

Assumes given timestamp is UTC and converts to given timezone.

```
>>> df = sqlContext.createDataFrame([('1997-02-28 10:30:00',)], ['t'])
>>> df.select(from_utc_timestamp(df.t, "PST").alias('t')).collect()
[Row(t=datetime.datetime(1997, 2, 28, 2, 30))]
```

New in version 1.5.

`pyspark.sql.functions.get_json_object(col, path)` [\[source\]](#)

Extracts json object from a json string based on json path specified, and returns json string of the extracted json object. It will return null if the input json string is invalid.

Parameters:

- **col** – string column in json format
- **path** – path to the json object to extract

```
>>> data = [{"1", '{"f1": "value1", "f2": "value2"}'}, {"2", '{"f1": "value1", "f2": "value2"}'}]
>>> df = sqlContext.createDataFrame(data, ("key", "jstring"))
>>> df.select(df.key, get_json_object(df.jstring, '$.f1').alias("c0"),
[Row(key=u'1', c0=u'value1', c1=u'value2'), Row(key=u'2', c0=u'value12', c1=None)]
```

New in version 1.6.

`pyspark.sql.functions.greatest(*cols)`

[\[source\]](#)

Returns the greatest value of the list of column names, skipping null values. This function takes at least 2 parameters. It will return null iff all parameters are null.

```
>>> df = sqlContext.createDataFrame([(1, 4, 3)], ['a', 'b', 'c'])
>>> df.select(greatest(df.a, df.b, df.c).alias("greatest")).collect()
[Row(greatest=4)]
```

New in version 1.5.

`pyspark.sql.functions.hex(col)`

[\[source\]](#)

Computes hex value of the given column, which could be StringType, BinaryType, IntegerType or LongType.

```
>>> sqlContext.createDataFrame([('ABC', 3)], ['a', 'b']).select(hex('a'), hex('b'))
[Row(hex(a)=u'414243', hex(b)=u'3')]
```

New in version 1.5.

`pyspark.sql.functions.hour(col)`

[\[source\]](#)

Extract the hours of a given date as integer.

```
>>> df = sqlContext.createDataFrame([('2015-04-08 13:08:15',)], ['a'])
>>> df.select(hour('a').alias('hour')).collect()
[Row(hour=13)]
```

New in version 1.5.

`pyspark.sql.functions.hypot(col1, col2)`

Computes $\sqrt{a^2 + b^2}$ without intermediate overflow or underflow.

New in version 1.4.

`pyspark.sql.functions.initcap(col)`

[\[source\]](#)

Translate the first letter of each word to upper case in the sentence.

```
>>> sqlContext.createDataFrame([('ab cd',)], ['a']).select(initcap("a").alias('v'))
[Row(v=u'Ab Cd')]
```

New in version 1.5.

`pyspark.sql.functions.input_file_name()`

[\[source\]](#)

Creates a string column for the file name of the current Spark task.

New in version 1.6.

`pyspark.sql.functions.instr(str, substr)`

[\[source\]](#)

Locate the position of the first occurrence of substr column in the given string. Returns null if either of the arguments are null.

NOTE: The position is not zero based, but 1 based index, returns 0 if substr could not be found in str.

```
>>> df = sqlContext.createDataFrame([('abcd',)], ['s',])
>>> df.select(instr(df.s, 'b').alias('s')).collect()
[Row(s=2)]
```

New in version 1.5.

`pyspark.sql.functions.isnan(col)`

[\[source\]](#)

An expression that returns true iff the column is NaN.

```
>>> df = sqlContext.createDataFrame([(1.0, float('nan')), (float('nan'), 2.0)], (
>>> df.select(isnan("a").alias("r1"), isnan(df.a).alias("r2")).collect()
[Row(r1=False, r2=False), Row(r1=True, r2=True)]
```

New in version 1.6.

`pyspark.sql.functions.isnull(col)`

[\[source\]](#)

An expression that returns true iff the column is null.

```
>>> df = sqlContext.createDataFrame([(1, None), (None, 2)], ("a", "b"))
>>> df.select(isnull("a").alias("r1"), isnull(df.a).alias("r2")).collect()
[Row(r1=False, r2=False), Row(r1=True, r2=True)]
```

New in version 1.6.

`pyspark.sql.functions.json_tuple(col, *fields)`

[\[source\]](#)

Creates a new row for a json column according to the given field names.

Parameters:

- **col** – string column in json format
- **fields** – list of fields to extract

```
>>> data = [("1", '{"f1": "value1", "f2": "value2"}'), ("2", '{"f1": "value
```



```
>>> df = sqlContext.createDataFrame(data, ("key", "jstring"))
>>> df.select(df.key, json_tuple(df.jstring, 'f1', 'f2')).collect()
[Row(key=u'1', c0=u'value1', c1=u'value2'), Row(key=u'2', c0=u'value12', c1=None)]
```

New in version 1.6.

`pyspark.sql.functions.kurtosis(col)`

Aggregate function: returns the kurtosis of the values in a group.

New in version 1.6.

`pyspark.sql.functions.lag(col, count=1, default=None)`

[\[source\]](#)

Window function: returns the value that is *offset* rows before the current row, and *defaultValue* if there is less than *offset* rows before the current row. For example, an *offset* of one will return the previous row at any given point in the window partition.

This is equivalent to the LAG function in SQL.

Parameters:

- **col** – name of column or expression
- **count** – number of row to extend
- **default** – default value

New in version 1.4.

`pyspark.sql.functions.last(col)`

Aggregate function: returns the last value in a group.

New in version 1.3.

`pyspark.sql.functions.last_day(date)`

[\[source\]](#)

Returns the last day of the month which the given date belongs to.

```
>>> df = sqlContext.createDataFrame([('1997-02-10',)], ['d'])
>>> df.select(last_day(df.d).alias('date')).collect()
[Row(date=datetime.date(1997, 2, 28))]
```

New in version 1.5.

`pyspark.sql.functions.lead(col, count=1, default=None)`

[\[source\]](#)

Window function: returns the value that is *offset* rows after the current row, and *defaultValue* if there is less than *offset* rows after the current row. For example, an *offset* of one will return the next row at any given point in the window partition.

This is equivalent to the LEAD function in SQL.

Parameters:

- **col** – name of column or expression
- **count** – number of row to extend
- **default** – default value

New in version 1.4.

`pyspark.sql.functions.least(*cols)` [\[source\]](#)

Returns the least value of the list of column names, skipping null values. This function takes at least 2 parameters. It will return null iff all parameters are null.

```
>>> df = sqlContext.createDataFrame([(1, 4, 3)], ['a', 'b', 'c'])
>>> df.select(least(df.a, df.b, df.c).alias("least")).collect()
[Row(least=1)]
```

New in version 1.5.

`pyspark.sql.functions.length(col)` [\[source\]](#)

Calculates the length of a string or binary expression.

```
>>> sqlContext.createDataFrame([('ABC',)], ['a']).select(length('a').alias('length')).collect()
[Row(length=3)]
```

New in version 1.5.

`pyspark.sql.functions.levenshtein(left, right)` [\[source\]](#)

Computes the Levenshtein distance of the two given strings.

```
>>> df0 = sqlContext.createDataFrame([('kitten', 'sitting',)], ['l', 'r'])
>>> df0.select(levenshtein('l', 'r').alias('d')).collect()
[Row(d=3)]
```

New in version 1.5.

`pyspark.sql.functions.lit(col)`

Creates a **Column** of literal value.

New in version 1.3.

`pyspark.sql.functions.locate(substr, str, pos=0)` [\[source\]](#)

Locate the position of the first occurrence of substr in a string column, after position pos.

NOTE: The position is not zero based, but 1 based index. returns 0 if substr could not be found in str.

Parameters:

- **substr** – a string
- **str** – a Column of StringType
- **pos** – start position (zero based)

```
>>> df = sqlContext.createDataFrame([('abcd',)], ['s',])
>>> df.select(locate('b', df.s, 1).alias('s')).collect()
[Row(s=2)]
```

New in version 1.5.

`pyspark.sql.functions.log(arg1, arg2=None)`

[\[source\]](#)

Returns the first argument-based logarithm of the second argument.

If there is only one argument, then this takes the natural logarithm of the argument.

```
>>> df.select(log(10.0, df.age).alias('ten')).map(lambda l: str(l.ten)[:7]).collect()
['0.30102', '0.69897']
```

```
>>> df.select(log(df.age).alias('e')).map(lambda l: str(l.e)[:7]).collect()
['0.69314', '1.60943']
```

New in version 1.5.

`pyspark.sql.functions.log10(col)`

Computes the logarithm of the given value in Base 10.

New in version 1.4.

`pyspark.sql.functions.log1p(col)`

Computes the natural logarithm of the given value plus one.

New in version 1.4.

`pyspark.sql.functions.log2(col)`

[\[source\]](#)

Returns the base-2 logarithm of the argument.

```
>>> sqlContext.createDataFrame([(4,)], ['a']).select(log2('a').alias('log2')).collect()
[Row(log2=2.0)]
```

New in version 1.5.

`pyspark.sql.functions.lower(col)`

Converts a string column to lower case.

New in version 1.5.

`pyspark.sql.functions.lpad(col, len, pad)`

[\[source\]](#)

Left-pad the string column to width *len* with *pad*.

```
>>> df = sqlContext.createDataFrame([('abcd',)], ['s',])
>>> df.select(lpad(df.s, 6, '#').alias('s')).collect()
[Row(s=u'##abcd')]
```

New in version 1.5.

`pyspark.sql.functions.ltrim(col)`

Trim the spaces from left end for the specified string value.

New in version 1.5.

`pyspark.sql.functions.max(col)`

Aggregate function: returns the maximum value of the expression in a group.

New in version 1.3.

`pyspark.sql.functions.md5(col)`

[\[source\]](#)

Calculates the MD5 digest and returns the value as a 32 character hex string.

```
>>> sqlContext.createDataFrame([('ABC',)], ['a']).select(md5('a').alias('hash')).  
[Row(hash=u'902fbdd2b1df0c4f70b4a5d23525e932')]
```

New in version 1.5.

`pyspark.sql.functions.mean(col)`

Aggregate function: returns the average of the values in a group.

New in version 1.3.

`pyspark.sql.functions.min(col)`

Aggregate function: returns the minimum value of the expression in a group.

New in version 1.3.

`pyspark.sql.functions.minute(col)`

[\[source\]](#)

Extract the minutes of a given date as integer.

```
>>> df = sqlContext.createDataFrame([('2015-04-08 13:08:15',)], ['a'])  
>>> df.select(minute('a').alias('minute')).collect()  
[Row(minute=8)]
```

New in version 1.5.

`pyspark.sql.functions.monotonicallyIncreasingId()`

[\[source\]](#)

Note: Deprecated in 1.6, use `monotonically_increasing_id` instead.

New in version 1.4.

`pyspark.sql.functions.monotonically_increasing_id()`

[\[source\]](#)

A column that generates monotonically increasing 64-bit integers.

The generated ID is guaranteed to be monotonically increasing and unique, but not consecutive. The current implementation puts the partition ID in the upper 31 bits, and the record number within each partition in the lower 33 bits. The assumption is that the data frame has less than 1 billion partitions, and each partition has less than 8 billion records.

As an example, consider a **DataFrame** with two partitions, each with 3 records. This expression would return the following IDs: 0, 1, 2, 8589934592 (1L << 33), 8589934593, 8589934594.

```
>>> df0 = sc.parallelize(range(2), 2).mapPartitions(lambda x: [(1,), (2,), (3,)])
>>> df0.select(monotonically_increasing_id().alias('id')).collect()
[Row(id=0), Row(id=1), Row(id=2), Row(id=8589934592), Row(id=8589934593), Row(id=
```

New in version 1.6.

`pyspark.sql.functions.month(col)`

[\[source\]](#)

Extract the month of a given date as integer.

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['a'])
>>> df.select(month('a').alias('month')).collect()
[Row(month=4)]
```

New in version 1.5.

`pyspark.sql.functions.months_between(date1, date2)`

[\[source\]](#)

Returns the number of months between date1 and date2.

```
>>> df = sqlContext.createDataFrame([('1997-02-28 10:30:00', '1996-10-30')], ['t', 'd'])
>>> df.select(months_between(df.t, df.d).alias('months')).collect()
[Row(months=3.9495967...)]
```

New in version 1.5.

`pyspark.sql.functions.nanvl(col1, col2)`

[\[source\]](#)

Returns col1 if it is not NaN, or col2 if col1 is NaN.

Both inputs should be floating point columns (DoubleType or FloatType).

```
>>> df = sqlContext.createDataFrame([(1.0, float('nan')), (float('nan'), 2.0)], ['a', 'b'])
>>> df.select(nanvl("a", "b").alias("r1"), nanvl(df.a, df.b).alias("r2")).collect()
[Row(r1=1.0, r2=1.0), Row(r1=2.0, r2=2.0)]
```

New in version 1.6.

`pyspark.sql.functions.next_day(date, dayOfWeek)`

[\[source\]](#)

Returns the first date which is later than the value of the date column.

Day of the week parameter is case insensitive, and accepts:

“Mon”, “Tue”, “Wed”, “Thu”, “Fri”, “Sat”, “Sun”.

```
>>> df = sqlContext.createDataFrame([('2015-07-27',)], ['d'])
>>> df.select(next_day(df.d, 'Sun').alias('date')).collect()
[Row(date=datetime.date(2015, 8, 2))]
```

New in version 1.5.

`pyspark.sql.functions.ntile(n)`

[\[source\]](#)

Window function: returns the ntile group id (from 1 to n inclusive) in an ordered window partition. For example, if n is 4, the first quarter of the rows will get value 1, the second quarter will get 2, the third quarter will get 3, and the last quarter will get 4.

This is equivalent to the NTILE function in SQL.

Parameters: n – an integer

New in version 1.4.

`pyspark.sql.functions.percentRank()`

Window function: .. note:: Deprecated in 1.6, use `percent_rank` instead.

New in version 1.6.

`pyspark.sql.functions.percent_rank()`

Window function: returns the relative rank (i.e. percentile) of rows within a window partition.

New in version 1.6.

`pyspark.sql.functions.pow(col1, col2)`

Returns the value of the first argument raised to the power of the second argument.

New in version 1.4.

`pyspark.sql.functions.quarter(col)`

[\[source\]](#)

Extract the quarter of a given date as integer.

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['a'])
>>> df.select(quarter('a').alias('quarter')).collect()
[Row(quarter=2)]
```

New in version 1.5.

`pyspark.sql.functions.rand(seed=None)`

[\[source\]](#)

Generates a random column with i.i.d. samples from $U[0.0, 1.0]$.

New in version 1.4.

`pyspark.sql.functions.randn(seed=None)` [\[source\]](#)

Generates a column with i.i.d. samples from the standard normal distribution.

New in version 1.4.

`pyspark.sql.functions.rank()`

Window function: returns the rank of rows within a window partition.

The difference between rank and denseRank is that denseRank leaves no gaps in ranking sequence when there are ties. That is, if you were ranking a competition using denseRank and had three people tie for second place, you would say that all three were in second place and that the next person came in third.

This is equivalent to the RANK function in SQL.

New in version 1.6.

`pyspark.sql.functions.regexp_extract(str, pattern, idx)` [\[source\]](#)

Extract a specific(idx) group identified by a java regex, from the specified string column.

```
>>> df = sqlContext.createDataFrame([('100-200',)], ['str'])
>>> df.select(regexp_extract('str', '(\d+)-(\d+)', 1).alias('d')).collect()
[Row(d=u'100')]
```

New in version 1.5.

`pyspark.sql.functions.regexp_replace(str, pattern, replacement)` [\[source\]](#)

Replace all substrings of the specified string value that match regexp with rep.

```
>>> df = sqlContext.createDataFrame([('100-200',)], ['str'])
>>> df.select(regexp_replace('str', '(\d+)', '--').alias('d')).collect()
[Row(d=u'-----')]
```

New in version 1.5.

`pyspark.sql.functions.repeat(col, n)` [\[source\]](#)

Repeats a string column n times, and returns it as a new string column.

```
>>> df = sqlContext.createDataFrame([('ab',)], ['s',])
>>> df.select(repeat(df.s, 3).alias('s')).collect()
[Row(s=u'ababab')]
```

New in version 1.5.

`pyspark.sql.functions.reverse(col)`

Reverses the string column and returns it as a new string column.

New in version 1.5.

`pyspark.sql.functions rint(col)`

Returns the double value that is closest in value to the argument and is equal to a mathematical integer.

New in version 1.4.

`pyspark.sql.functions.round(col, scale=0)`

[\[source\]](#)

Round the value of *e* to *scale* decimal places if *scale* ≥ 0 or at integral part when *scale* < 0 .

```
>>> sqlContext.createDataFrame([(2.546,)], ['a']).select(round('a', 1).alias('r'))  
[Row(r=2.5)]
```

New in version 1.5.

`pyspark.sql.functions.rowNumber()`

Window function: .. note:: Deprecated in 1.6, use `row_number` instead.

New in version 1.6.

`pyspark.sql.functions.row_number()`

Window function: returns a sequential number starting at 1 within a window partition.

New in version 1.6.

`pyspark.sql.functions.rpad(col, len, pad)`

[\[source\]](#)

Right-pad the string column to width *len* with *pad*.

```
>>> df = sqlContext.createDataFrame([('abcd',)], ['s',])  
>>> df.select(rpad(df.s, 6, '#').alias('s')).collect()  
[Row(s=u'abcd##')]
```

New in version 1.5.

`pyspark.sql.functions.rtrim(col)`

Trim the spaces from right end for the specified string value.

New in version 1.5.

`pyspark.sql.functions.second(col)`

[\[source\]](#)

Extract the seconds of a given date as integer.

```
>>> df = sqlContext.createDataFrame([('2015-04-08 13:08:15',)], ['a'])  
>>> df.select(second('a').alias('second')).collect()  
[Row(second=15)]
```


New in version 1.5.

`pyspark.sql.functions.sha1(col)`

[\[source\]](#)

Returns the hex string result of SHA-1.

```
>>> sqlContext.createDataFrame([('ABC',)], ['a']).select(sha1('a').alias('hash'))
[Row(hash=u'3c01bdbb26f358bab27f267924aa2c9a03fcfdb8')]
```

New in version 1.5.

`pyspark.sql.functions.sha2(col, numBits)`

[\[source\]](#)

Returns the hex string result of SHA-2 family of hash functions (SHA-224, SHA-256, SHA-384, and SHA-512). The numBits indicates the desired bit length of the result, which must have a value of 224, 256, 384, 512, or 0 (which is equivalent to 256).

```
>>> digests = df.select(sha2(df.name, 256).alias('s')).collect()
>>> digests[0]
Row(s=u'3bc51062973c458d5a6f2d8d64a023246354ad7e064b1e4e009ec8a0699a3043')
>>> digests[1]
Row(s=u'cd9fb1e148ccd8442e5aa74904cc73bf6fb54d1d54d333bd596aa9bb4bb4e961')
```

New in version 1.5.

`pyspark.sql.functions.shiftLeft(col, numBits)`

[\[source\]](#)

Shift the the given value numBits left.

```
>>> sqlContext.createDataFrame([(21,)], ['a']).select(shiftLeft('a', 1).alias('r'))
[Row(r=42)]
```

New in version 1.5.

`pyspark.sql.functions.shiftRight(col, numBits)`

[\[source\]](#)

Shift the the given value numBits right.

```
>>> sqlContext.createDataFrame([(42,)], ['a']).select(shiftRight('a', 1).alias('r'))
[Row(r=21)]
```

New in version 1.5.

`pyspark.sql.functions.shiftRightUnsigned(col, numBits)`

[\[source\]](#)

Unsigned shift the the given value numBits right.

```
>>> df = sqlContext.createDataFrame([(-42,)], ['a'])
>>> df.select(shiftRightUnsigned('a', 1).alias('r')).collect()
```

```
[Row(r=9223372036854775787)]
```

New in version 1.5.

`pyspark.sql.functions.signum(col)`

Computes the signum of the given value.

New in version 1.4.

`pyspark.sql.functions.sin(col)`

Computes the sine of the given value.

New in version 1.4.

`pyspark.sql.functions.sinh(col)`

Computes the hyperbolic sine of the given value.

New in version 1.4.

`pyspark.sql.functions.size(col)`

[\[source\]](#)

Collection function: returns the length of the array or map stored in the column.

Parameters: `col` – name of column or expression

```
>>> df = sqlContext.createDataFrame([(1, 2, 3)], ([1]), ([]), ['data'])
>>> df.select(size(df.data)).collect()
[Row(size(data)=3), Row(size(data)=1), Row(size(data)=0)]
```

New in version 1.5.

`pyspark.sql.functions.skewness(col)`

Aggregate function: returns the skewness of the values in a group.

New in version 1.6.

`pyspark.sql.functions.sort_array(col, asc=True)`

[\[source\]](#)

Collection function: sorts the input array for the given column in ascending order.

Parameters: `col` – name of column or expression

```
>>> df = sqlContext.createDataFrame([(2, 1, 3)], ([1]), ([]), ['data'])
>>> df.select(sort_array(df.data).alias('r')).collect()
[Row(r=[1, 2, 3]), Row(r=[1]), Row(r=[])]
>>> df.select(sort_array(df.data, asc=False).alias('r')).collect()
[Row(r=[3, 2, 1]), Row(r=[1]), Row(r=[])]
```

New in version 1.5.

`pyspark.sql.functions.soundex(col)`

[\[source\]](#)

Returns the SoundEx encoding for a string

```
>>> df = sqlContext.createDataFrame([("Peters",), ("Uhrbach",)], ['name'])
>>> df.select(soundex(df.name).alias("soundex")).collect()
[Row(soundex=u'P362'), Row(soundex=u'U612')]
```

New in version 1.5.

pyspark.sql.functions.**sparkPartitionId()**

[\[source\]](#)

Note: Deprecated in 1.6, use `spark_partition_id` instead.

New in version 1.4.

pyspark.sql.functions.**spark_partition_id()**

[\[source\]](#)

A column for partition ID of the Spark task.

Note that this is indeterministic because it depends on data partitioning and task scheduling.

```
>>> df.repartition(1).select(spark_partition_id().alias("pid")).collect()
[Row(pid=0), Row(pid=0)]
```

New in version 1.6.

pyspark.sql.functions.**split(str, pattern)**

[\[source\]](#)

Splits str around pattern (pattern is a regular expression).

NOTE: pattern is a string represent the regular expression.

```
>>> df = sqlContext.createDataFrame([('ab12cd',)], ['s',])
>>> df.select(split(df.s, '[0-9]+').alias('s')).collect()
[Row(s=[u'ab', u'cd'])]
```

New in version 1.5.

pyspark.sql.functions.**sqrt(col)**

Computes the square root of the specified float value.

New in version 1.3.

pyspark.sql.functions.**stddev(col)**

Aggregate function: returns the unbiased sample standard deviation of the expression in a group.

New in version 1.6.

pyspark.sql.functions.**stddev_pop(col)**

Aggregate function: returns population standard deviation of the expression in a group.

New in version 1.6.

`pyspark.sql.functions.stddev_samp(col)`

Aggregate function: returns the unbiased sample standard deviation of the expression in a group.

New in version 1.6.

`pyspark.sql.functions.struct(*cols)`

[\[source\]](#)

Creates a new struct column.

Parameters: `cols` – list of column names (string) or list of `column` expressions

```
>>> df.select(struct('age', 'name').alias("struct")).collect()
[Row(struct=Row(age=2, name=u'Alice')), Row(struct=Row(age=5, name=u'Bob'))]
>>> df.select(struct([df.age, df.name]).alias("struct")).collect()
[Row(struct=Row(age=2, name=u'Alice')), Row(struct=Row(age=5, name=u'Bob'))]
```

New in version 1.4.

`pyspark.sql.functions.substring(str, pos, len)`

[\[source\]](#)

Substring starts at `pos` and is of length `len` when `str` is String type or returns the slice of byte array that starts at `pos` in byte and is of length `len` when `str` is Binary type

```
>>> df = sqlContext.createDataFrame([('abcd',)], ['s',])
>>> df.select(substring(df.s, 1, 2).alias('s')).collect()
[Row(s=u'ab')]
```

New in version 1.5.

`pyspark.sql.functions.substring_index(str, delim, count)`

[\[source\]](#)

Returns the substring from string `str` before `count` occurrences of the delimiter `delim`. If `count` is positive, everything the left of the final delimiter (counting from left) is returned. If `count` is negative, every to the right of the final delimiter (counting from the right) is returned. `substring_index` performs a case-sensitive match when searching for `delim`.

```
>>> df = sqlContext.createDataFrame([('a.b.c.d',)], ['s'])
>>> df.select(substring_index(df.s, '.', 2).alias('s')).collect()
[Row(s=u'a.b')]
>>> df.select(substring_index(df.s, '.', -3).alias('s')).collect()
[Row(s=u'b.c.d')]
```

New in version 1.5.

`pyspark.sql.functions.sum(col)`

Aggregate function: returns the sum of all values in the expression.

New in version 1.3.

`pyspark.sql.functions.sumDistinct(col)`

Aggregate function: returns the sum of distinct values in the expression.

New in version 1.3.

`pyspark.sql.functions.tan(col)`

Computes the tangent of the given value.

New in version 1.4.

`pyspark.sql.functions.tanh(col)`

Computes the hyperbolic tangent of the given value.

New in version 1.4.

`pyspark.sql.functions.toDegrees(col)`

Converts an angle measured in radians to an approximately equivalent angle measured in degrees.

New in version 1.4.

`pyspark.sql.functions.toRadians(col)`

Converts an angle measured in degrees to an approximately equivalent angle measured in radians.

New in version 1.4.

`pyspark.sql.functions.to_date(col)`

[\[source\]](#)

Converts the column of StringType or TimestampType into DateType.

```
>>> df = sqlContext.createDataFrame([('1997-02-28 10:30:00',)], ['t'])
>>> df.select(to_date(df.t).alias('date')).collect()
[Row(date=datetime.date(1997, 2, 28))]
```

New in version 1.5.

`pyspark.sql.functions.to_utc_timestamp(timestamp, tz)`

[\[source\]](#)

Assumes given timestamp is in given timezone and converts to UTC.

```
>>> df = sqlContext.createDataFrame([('1997-02-28 10:30:00',)], ['t'])
>>> df.select(to_utc_timestamp(df.t, "PST").alias('t')).collect()
[Row(t=datetime.datetime(1997, 2, 28, 18, 30))]
```

New in version 1.5.

`pyspark.sql.functions.translate(srcCol, matching, replace)`

[\[source\]](#)

A function translate any character in the *srcCol* by a character in *matching*. The characters in *replace* is corresponding to the characters in *matching*. The translate will happen when any character in the string matching with the character in the *matching*.

```
>>> sqlContext.createDataFrame([('translate',)], ['a']).select(translate('a', "rn
[Row(r=u'1a2s3ae')]
```

New in version 1.5.

`pyspark.sql.functions.trim(col)`

Trim the spaces from both ends for the specified string column.

New in version 1.5.

`pyspark.sql.functions.trunc(date, format)`

[\[source\]](#)

Returns date truncated to the unit specified by the format.

Parameters: **format** – ‘year’, ‘YYYY’, ‘yy’ or ‘month’, ‘mon’, ‘mm’

```
>>> df = sqlContext.createDataFrame([('1997-02-28',)], ['d'])
>>> df.select(trunc(df.d, 'year').alias('year')).collect()
[Row(year=datetime.date(1997, 1, 1))]
>>> df.select(trunc(df.d, 'mon').alias('month')).collect()
[Row(month=datetime.date(1997, 2, 1))]
```

New in version 1.5.

`pyspark.sql.functions.udf(f, returnType=StringType)`

[\[source\]](#)

Creates a **Column** expression representing a user defined function (UDF).

```
>>> from pyspark.sql.types import IntegerType
>>> slen = udf(lambda s: len(s), IntegerType())
>>> df.select(slen(df.name).alias('slen')).collect()
[Row(slen=5), Row(slen=3)]
```

New in version 1.3.

`pyspark.sql.functions.unbase64(col)`

Decodes a BASE64 encoded string column and returns it as a binary column.

New in version 1.5.

`pyspark.sql.functions.unhex(col)`

[\[source\]](#)

Inverse of hex. Interprets each pair of characters as a hexadecimal number and converts to the byte representation of number.

```
>>> sqlContext.createDataFrame([('414243',)], ['a']).select(unhex('a')).collect()
[Row(unhex(a)=bytearray(b'ABC'))]
```

New in version 1.5.

`pyspark.sql.functions.unix_timestamp(timestamp=None, format='yyyy-MM-dd HH:mm:ss')`
Convert time string with given pattern ('yyyy-MM-dd HH:mm:ss', by default) to Unix [\[source\]](#) time stamp (in seconds), using the default timezone and the default locale, return null if fail.
if *timestamp* is None, then it returns current timestamp.

New in version 1.5.

`pyspark.sql.functions.upper(col)`
Converts a string column to upper case.

New in version 1.5.

`pyspark.sql.functions.var_pop(col)`
Aggregate function: returns the population variance of the values in a group.

New in version 1.6.

`pyspark.sql.functions.var_samp(col)`
Aggregate function: returns the unbiased variance of the values in a group.

New in version 1.6.

`pyspark.sql.functions.variance(col)`
Aggregate function: returns the population variance of the values in a group.

New in version 1.6.

`pyspark.sql.functions.weekofyear(col)` [\[source\]](#)
Extract the week number of a given date as integer.

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['a'])
>>> df.select(weekofyear(df.a).alias('week')).collect()
[Row(week=15)]
```

New in version 1.5.

`pyspark.sql.functions.when(condition, value)` [\[source\]](#)
Evaluates a list of conditions and returns one of multiple possible result expressions. If `Column.otherwise()` is not invoked, None is returned for unmatched conditions.

Parameters:

- **condition** – a boolean `Column` expression.
- **value** – a literal value, or a `Column` expression.

```
>>> df.select(when(df['age'] == 2, 3).otherwise(4).alias("age")).collect()
[Row(age=3), Row(age=4)]
```

```
>>> df.select(when(df.age == 2, df.age + 1).alias("age")).collect()
[Row(age=3), Row(age=None)]
```

New in version 1.4.

`pyspark.sql.functions.year(col)`

[\[source\]](#)

Extract the year of a given date as integer.

```
>>> df = sqlContext.createDataFrame([('2015-04-08',)], ['a'])
>>> df.select(year('a').alias('year')).collect()
[Row(year=2015)]
```

New in version 1.5.